



GeDefense WP

OPEN CORE 8.1.0

Руководство по администрированию, безопасности и архитектуре

Установка | Эксплуатация | Модули | Recovery | Технический план

VisionGaiaTechnology

Состояние документации: 02.09.2026

Основа: предоставленный исходный код GeDefense WP - Open Core 8.1.0

Содержание

Руководство разделено на пять частей. Часть IV содержит полную техническую архитектурную карту из `ARCHITECTURE.md`.

Часть I - Установка и первичная конфигурация

- Системные требования
- Установка
- Setup Wizard - семь шагов
- Рекомендуемый ввод в эксплуатацию в три этапа

Часть II - Руководство администратора по защитным модулям

- AEGIS
- ZEUS
- CERBERUS
- PROMETHEUS
- TRINITY XDR
- MORPHEUS
- TITAN
- HADES
- AIRLOCK
- GHOST TRAP
- NEMESIS
- STYX
- CHRONOS
- Integrity Monitor & Scanner
- KEY VAULT
- THRONEGUARD
- LOGINPAGER
- ORACLE & Security Center
- GORGON
- Secure Downloads

Часть III - Эксплуатация, обслуживание и диагностика

- Проверка после изменений
- Типичные неисправности
- Backup & Recovery
- Защита данных и телеметрия

Часть IV - Полный технический план

- Обзор системы
- Глобальное дерево архитектуры
- Привязка файлов
- Документация модулей
- Dashboard
- CSS/UI
- API
- Потоки данных
- Shared/Core
- Архитектурные связи
- Ссылки на файлы
- Архитектурные замечания

Часть V - Источники и версия

Статус документа и рекомендации по чтению

Это руководство описывает **GeDefense WP - Open Core 8.1.0** на основе предоставленного исходного кода плагина и соответствующего файла `ARCHITECTURE.md`. В одном документе объединены три точки зрения:

- 1. Установка и безопасный первичный запуск** - что необходимо сделать после установки и в какой последовательности.
- 2. Руководство администратора** - что технически делают основные разделы Dashboard и настройки, когда их целесообразно использовать и где ошибочная конфигурация может нарушить работу сайта.
- 3. Технический план системы** - полная архитектурная карта с файлами, модулями, связями с Dashboard, API, потоками данных и зависимостями.

Важно: В ряде мест GeDefense намеренно работает по принципу fail-closed и использует строгие механизмы блокировки. Такие функции, как Hades, Titan, Zeus, ThroneGuard, Morpheus Enforcement или Styx, могут блокировать легитимные обращения и интеграции, если включить их без предварительного тестирования и этапа допуска. Перед изменениями в рабочей среде необходимо иметь рабочую резервную копию и альтернативный административный путь доступа.

Условные обозначения документа

Обозначение	Значение
Рекомендуется для старта	консервативная исходная настройка для первичного ввода в эксплуатацию
Проверить совместимость	может влиять на темы, плагины, REST-endpoint'ы, внешние API или процессы входа
Риск блокировки доступа	может заблокировать доступ самому администратору
Расширенная настройка	для администраторов, которые понимают техническое воздействие и способны его протестировать
Источник: код	утверждение выведено из текущей реализации плагина либо интерфейса Dashboard

Часть I - Установка и первичная конфигурация

1. Системные требования

В заголовке предоставленной версии плагина указано **GeDefense WP - Open Core 8.1.0**. В README проекта указаны WordPress 6.0+ и PHP 8.1-8.4. Базовая архитектура не зависит от внешних PHP-пакетов: ключевые функции загружаются из самого плагина.

Перед развертыванием на практике следует обеспечить следующее:

- административный доступ к WordPress с правом `manage_options`;
- право записи WordPress в предусмотренную область Vault/Upload;
- рабочий HTTPS, особенно если используются HSTS, защищенные cookie, Sandbox Origin или Admission Tokens;
- резервная копия файлов и базы данных;
- при использовании Reverse Proxy/CDN - корректное определение Client IP, чтобы whitelist'ы и блокировки применялись к фактическому IP администратора;
- для необязательных AI-функций - собственный Groq API Key, сохраняемый через Key Vault.

2. Установка

1. Разместить каталог плагина в `wp-content/plugins/` или установить ZIP через WordPress.
2. Активировать **GeDefense WP - Open Core**.
3. При активации `class-vis-schema.php` создает необходимую схему базы данных. При административном запросе GeDefense проверяет версии и при необходимости повторно выполняет миграции схемы.
4. В панели WordPress появляется **VGT Suite**. Пока мастер первоначальной настройки не завершен, GeDefense показывает уведомление с предложением запустить Setup Wizard.
5. Полностью пройти Setup Wizard до ручного включения агрессивных профилей защиты или маскировки login-пути.

Что происходит при загрузке

Точка входа `gedefense-wp.php` загружает Bootstrapper и Security Core, подключает Schema/Vault и запускает две фазы защиты. **Phase 1** активируется немедленно и покрывает ранний периметр; **Phase 2** загружается в обычном жизненном цикле плагина и подключает Dashboard и дополнительные защитные модули. Полный технический процесс описан в Части IV.

3. Setup Wizard - семь шагов

Текущий Setup Wizard является не просто визуальным введением, а записывает реальные параметры модулей. Кроме того, он определяет IP администратора через тот же проху-aware resolver, что и защитные модули, и первоначально заносит его в поля AEGIS и Prometheus, если они еще пусты.

Шаг 1 - Проверка системы и IP whitelist

Цель: предотвратить ситуацию, когда собственная административная сессия будет распознана AEGIS или Prometheus как атакующая.

Поле	Действие
AEGIS & CERBERUS IP-Whitelist	доверенные IP; Wizard описывает их как исключенные из Firewall/Ban System
Prometheus Scanner Whitelist	IP, активность которых не должна оцениваться тем же эвристическим способом

Рекомендация: проверить обнаруженный IP. При VPN, Cloudflare, Reverse Proxy или динамическом адресе не принимать значение автоматически, а убедиться, что GeDefense действительно видит требуемый Client IP.

Риск блокировки доступа: сам Dashboard предупреждает, что отсутствие разрешений AEGIS и Prometheus может блокировать REST-, Nonce- и Update-запросы.

Шаг 2 - Firewall и WAF

Если явная конфигурация еще отсутствует, Wizard по умолчанию включает AEGIS, Zeus и Cerberus.

AEGIS запускается в режиме `learning` по умолчанию. Learning журналирует и наблюдает; `strict` - агрессивная Zero-Trust реакция с прерыванием запроса и автоматической логикой блокировки.

Zeus в Wizard предварительно выбирается с базовым Firewall и 6G Blacklist. Zeus работает раньше более сложных слоев WordPress и проверяет структурные границы запросов, Admission, Host Lock, Budgets и Route Contracts.

Cerberus управляет быстрым уровнем IP-блокировок и является конечной точкой эскалации для нескольких других модулей.

Рекомендуется для старта: AEGIS Learning, Zeus Balanced/Default, Cerberus включен. AEGIS Strict или более жесткие профили Zeus следует применять только после проверки логов и whitelist'ов.

Шаг 3 - Защита от вредоносного ПО и RASP

Prometheus изначально включен в Wizard. Он оценивает поведение, частоту запросов и Threat Scores.

Morpheus также изначально включен, но его **Enforcement Mode** не активируется. Это важное различие: Audit/Learning сначала позволяет наблюдать Callstack'и и легитимные операции. Строгое принудительное применение следует включать только после проверки совместимости.

Шаг 4 - Обман и ловушки

Nemesis и **Ghost Trap** изначально включены в Wizard. Nemesis использует механизмы Deception и Canary; Ghost Trap предоставляет Honeypot-/Decoy-маршруты и приманки, обращение к которым может вызвать защитную реакцию.

Проверить совместимость: собственные Security Scanner'ы, Monitoring Services или внешние Penetration Tests могут намеренно обращаться к таким Decoy-ресурсам. Авторизованные сканеры следует учитывать заранее.

Шаг 5 - Hardening и Stealth

Titan изначально активен. Wizard по умолчанию включает защиту XML-RPC и скрывание версии; блокировка REST и отключение Feed автоматически не включаются.

Hades по умолчанию в Wizard выключен. При активации мастер задает секретный параметр и Secret. После сохранения он показывает сформированный секретный URL администратора.

ThroneGuard и **LoginPager** также не включаются автоматически. Это обоснованно: ThroneGuard изменяет привилегии, а LoginPager - интерфейс входа.

Шаг 6 - Autopilot, Egress и AI

Chronos и **Styx** изначально включены. Chronos выполняет плановые проверки целостности и файловой системы. Styx контролирует исходящую HTTP-коммуникацию через whitelist.

При необходимости можно указать **Groq API Key**. Wizard описывает его хранение как зашифрованное в Vault. AI-функции являются дополнительной диагностикой и не заменяют детерминированные защитные слои.

Шаг 7 - Ввод защиты в действие

На последнем шаге Wizard сводит выбранные защитные модули и сохраняет конфигурацию. Если активен Hades или login-путь Zeus, экран успешного завершения показывает новые способы административного доступа.

Сразу после этого:

1. Сохранить новый административный путь доступа в менеджере паролей.
2. Проверить сайт в приватном окне браузера.
3. Проверить Frontend, Login, загрузку медиа, REST/AJAX-функции и запланированные внешние API.
4. Проверить Security Center и логи на неожиданные блокировки.
5. Только после этого ужесточать Strict-/Enforcement-профили.

4. Рекомендуемый ввод в эксплуатацию в три этапа

Этап А - Baseline

- AEGIS: **Learning**
- Zeus: **Balanced**, без экспериментов с Incident Lockdown
- Cerberus: включен
- Prometheus: включен, собственный Admin IP разрешен
- Titan: включен, сначала совместимая Policy
- Morpheus: Audit/Learning, Enforcement выключен
- Chronos: включен
- Key Vault: управлять Secrets только через Vault

Этап В - Наблюдение и совместимость

Следует протестировать как минимум типичные рабочие процессы: вход, редактирование страниц, загрузку медиа, обновления плагинов/тем, WooCommerce/REST при наличии, Webhooks, внешние API Calls, Cron и ПО резервного копирования. Блокировки следует устранять не полным отключением модуля, а минимально необходимым исключением.

Этап С - Hardening

После получения стабильной Baseline можно поэтапно включать AEGIS Strict, более жесткие профили Zeus, Titan Enforcement, Morpheus Enforcement, Hades и ThroneGuard. За один раз следует менять только одну критичную для безопасности переменную, а затем повторно проверять основные функции.

Часть II - Руководство администратора по защитным модулям

5. AEGIS - Deep Packet Inspection WAF

Назначение

AEGIS анализирует входящие данные из GET, POST, JSON, заголовков, cookie и других областей запроса. Архитектура описывает рекурсивную нормализацию и классы атак, включая SQL Injection, XSS, RCE, LFI и небезопасную десериализацию. Dashboard показывает состояние WAF, Protection Mode, whitelist'ы и диагностику Oracle.

Основные настройки

Настройка	Значение	Рекомендация
Engine активен	включает/выключает AEGIS	включить

Настройка	Значение	Рекомендация
Learning	журналирует/наблюдает вместо немедленной максимальной эскалации	первичный запуск
Strict	агрессивная реакция, прерывание Request/Ban coupling	после Baseline
Trusted IPs	IP, считающиеся административными/доверенными	проверить собственный Admin IP
Trusted User-Agents	исключения по UA keyword	использовать экономно
Oracle Neural Link	необязательная AI-диагностика	только с Vault Key

Эксплуатационные рекомендации

Whitelist'ы критичны для безопасности: слишком широкие создают Bypass, слишком узкие могут блокировать легитимные административные действия. При Proxy/CDN сначала всегда проверяйте разрешение Client IP. По данным UI, Groq Ping в Dashboard проверяет TLS/Auth/API latency без передачи Prompt или содержимого.

6. ZEUS - Pre-Boot Admission Kernel

Назначение

Zeus контролирует запросы на максимально раннем этапе. В текущей версии сюда входят Host- и Canonical Routing, структурные границы Request, Route Contracts, Request Budgets, Admission Tokens, Lockdown States и tamper-evident Blackbox.

Security Profile

Профиль	Характеристика
BALANCED	рекомендуемый исходный баланс
STRICT	более сильная защита, более высокий риск несовместимости
PARANOID	максимальное сдерживание, только после тестов
CUSTOM	политика, заданная вручную

Emergency Lockdown State

NORMAL, HARDENED, FORTRESS и INCIDENT LOCKDOWN последовательно повышают требования к доступу. Incident Lockdown следует воспринимать как аварийный режим, а не как постоянную стандартную конфигурацию.

Host Lock

- **DISABLED:** Host Policy отсутствует.
- **AUDIT:** нарушения только журналируются.
- **REJECT:** неканонические Host'ы отклоняются с кодом 421.
- **Canonical Hosts:** по одному разрешенному Host в строке.

Проверить совместимость: необходимо учитывать Staging Domains, альтернативные домены, внутренние Health Check'и и Hostname'ы Proxy.

Request Budget Engine

Стандартные значения UI: 180 Requests/60s на IP и 450 Requests/60s на подсеть. В качестве действий доступны Throttle, временный 503 Reject или XDR Signal. Эти пороги не являются универсальными: магазины, API, Crawling и сайты с интенсивным AJAX могут требовать иных профилей.

Brute-Force и защита Login

UI предоставляет Custom Login URL Slug, Magic Gateway Cookie, 404-Sweep-Lockout, Failed-Login-Lockout и принудительные Admin Session Timeouts. Изменения путей входа всегда следует тестировать во втором браузере и документировать.

Request Envelope Limits

Стандартные значения UI включают, в частности, длину Query 2048 байт, 100 Query Parameters, 50 Headers, суммарный размер Headers 16 KiB и Cookie 8 KiB. В форме предварительно установлен Default POST Limit 64 MiB. Route Contracts позволяют дополнительно применять более точные правила для отдельных путей.

Blackbox и Hardening Lab

Blackbox собирает раннюю телеметрию Zeus в Hash Chain и может передавать ее в TRINITY XDR. Hardening Lab предоставляет Benchmarks и Self-Tests; показатели производительности всегда зависят от Host и реальной нагрузки.

7. CERBERUS - Периметр и Ban Layer

Cerberus является быстрым уровнем блокировки. Архитектура связывает его с APCu/Object Cache lookup'ами, таблицей Ban и экспортом правил для Webserver/Firewall. Блокировки могут инициироваться AEGIS, Prometheus и Ghost Trap.

Действия в Dashboard: просмотр заблокированных IP, ручная блокировка, снятие блокировки и экспорт правил. Перед Unban необходимо проверить причину в Event-/XDR-контексте.

8. PROMETHEUS - Behavioral Profiler

Prometheus оценивает не только отдельные Payload'ы, но и поведение во времени. В Dashboard настраиваются Score Decay, 404-/Auth-/Burst-/Method-/Parameter-/Regex Penalties и пороги Event Horizon.

Рекомендации

- Слишком низкие пороги: False Positives для администраторов, Crawlers и API.
- Слишком высокие пороги: поведенческое обнаружение теряет эффективность.
- Изменения следует выполнять на основе реальной телеметрии.
- Admin IP и известные Monitoring IP следует корректно добавить в whitelist.

9. TRINITY XDR - Корреляция и Response

TRINITY объединяет события различных сенсоров в Incidents и Responses. Dashboard разделен на Situation View, Incidents, Attack Stories, Entities, Responses, Evidence, Policy, Events и Health.

Основные области Policy

- включение/выключение автоматического Response;
- Containment Policy;
- эскалация для повторных нарушителей;
- срок Retention;
- TTL для Actor Ban, Subnet, Zeus Route, Admission, Morpheus Overlay и Styx Overlay.

Принцип: временное и отслеживаемое сдерживание в нормальной эксплуатации безопаснее необоснованно постоянных блокировок. Перед ручной эскалацией следует проверять Evidence и историю Response.

10. MORPHEUS - Runtime Application Self-Protection

Morpheus контролирует контексты PHP-вызовов и защищает чувствительные операции во время выполнения. Архитектура указывает DB Shields для wp_users/wp_usermeta, Network Shield против Cloud Metadata/SSRF и Path Jail.

Сначала Audit, затем Enforcement

Setup Wizard не включает Morpheus Enforcement автоматически. Так и следует оставлять систему, пока не будут наблюдаемы типичные плагины и рабочие процессы. В Dashboard записи Audit Matrix и AI-анализ Callstack можно одобрять или отклонять.

Проверить совместимость: плагины, которые легитимно изменяют пользователей, Usermeta, чувствительные настройки или обращаются к необычным сетевым целям, могут срабатывать в строгом режиме.

11. TITAN - Application & Browser Confinement

Titan - один из самых обширных модулей. Он объединяет Hardening WordPress с HTTP-/Browser Policies и жизненным циклом Assurance.

Policy Profiles

Профиль	Оценка UI	Применение
Compatible	Low Breakage Risk	старт / неоднородная среда плагинов
Balanced	Medium Breakage Risk	стандарт после проверки совместимости

Профиль	Оценка UI	Применение
Strict	High Breakage Risk	контролируемые среды
Paranoid	Very High Breakage Risk	специальные случаи / лаборатория
Custom	Admin Defined	целевая собственная Policy

CSP и Origins

Titan ведет отдельные Origins для Script, Style, Image, Connect и Frame. Чем строже CSP и списки Origins, тем выше вероятность поломки внешних Fonts, платежных провайдеров, карт, Analytics, Video Embeds или API Clients. Поэтому Titan поддерживает Learning/Observed Origins и процесс Candidate -> Report-Only -> Enforce.

Assurance Lifecycle

UI проводит через **Learning -> Candidate -> Validate Candidate -> Report-Only -> Enforce** и предоставляет Rollback. Это рекомендуемый путь: неизвестную CSP не следует сразу переводить в Enforcement.

HSTS

- **HSTS enabled**: отправляет Strict-Transport-Security только при SSL.
- **max-age**: код ограничивает значение диапазоном от 300 до 63 072 000 секунд.
- **includeSubDomains**: распространяет правило на Subdomains.
- **preload**: добавляется только совместно с includeSubDomains.

Осторожно: HSTS, особенно includeSubDomains и preload, влияет на инфраструктуру за пределами WordPress. Включайте только тогда, когда все затрагиваемые Host'ы постоянно поддерживают HTTPS.

Permissions Policy

Permissions allowed for self определяет, какие функции браузера разрешены собственной Origin. Ненужные функции не следует включать без необходимости.

XML-RPC

Реализация поддерживает несколько режимов: **disabled**, **pingback_disabled**, **auth_only**, **honeypot** и **custom**. Legacy-переключатель **titan_block_xmlrpc** внутри системы преобразуется в соответствующую Policy.

- **disabled**: полностью отключает XML-RPC.
- **pingback_disabled**: удаляет Pingback Methods, сохраняя остальные варианты использования XML-RPC.
- **honeypot**: фиксирует обращения к **xmlrpc.php** как Deception Event и завершает их с кодом 403.

REST API и User Enumeration

titan_anti_enum удаляет публичные User Endpoints и блокирует Author Enumeration. **titan_block_rest** ограничивает REST-доступ для неавторизованных пользователей, но содержит список разрешенных Namespaces. Эта функция может затрагивать Headless Frontends, формы, WooCommerce, внешние интеграции и собственные REST API.

Application Passwords

Реализация поддерживает **allow**, **disable** и **audit**. В режиме Audit аутентификация разрешается, а событие отправляется в XDR; **Disable** отключает WordPress Application Passwords.

Server Spoofing и Camouflage

Titan может изменять Server Headers и имитировать Generator Meta для Laravel, Drupal или Joomla. Это Reconnaissance Deception, а не замена обновлениям и Patching. При наличии внешнего Webserver/Proxy итоговый Header может быть перезаписан вне PHP.

Cleanup Options

- удалить версию WordPress;
- удалить Discovery Links;
- удалить Emoji Infrastructure;
- сократить oEmbed Discovery;
- удалить Asset-ver Querystrings;
- отключить Heartbeat.

Heartbeat следует отключать только после проверки функций Editor/Plugins, которые от него зависят.

Fetch Metadata и Browser Policies

Titan может учитывать Fetch Metadata Signals и дополнительные Browser Security Headers в Policy. UI явно различает Support/Observation и реальную гарантию глобального Enforcement.

Active Content Sandbox

Зарегистрированный HTML-, SVG- и XML-контент может отображаться через подписанные Preview URL с Sandbox CSP без Scripts. Выделенная Sandbox Origin принимается только в том случае, если она использует HTTPS, отделена от Application Host и не получает привилегии через общую WordPress Cookie Domain.

Server Rules

Согласно UI, Nginx Rules записываются только в проверенный приватный Vault за пределами Document Root; в противном случае они остаются лишь Export. **GENERATED/EXPORT ONLY не означает VERIFIED ACTIVE.** Активацию на Webserver необходимо проверять отдельно.

12. HADES - Admin Cloaking

Hades скрывает стандартные Admin-/Login Paths за секретным Gate и может выдавать 404 Mimicry.

Настройки

- включить Engine;
- Admin Parameter, например vgt_access;
- Secret, например omega в Wizard;
- дополнительные Stealth-/Server Rules в зависимости от версии Dashboard.

Риск блокировки доступа: после активации сохраните сформированный секретный Admin URL и немедленно проверьте его в приватном браузере. Parameter и Secret являются учетными данными доступа и не должны попадать в Tickets, Screenshots или публичную документацию.

13. AIRLOCK - Защита Upload

Airlock подключается к WordPress Upload Path, проверяет File Types/Magic Bytes, может очищать SVG/XML и передавать подозрительные файлы в Scanner/Quarantine.

Настройка	Действие
Engine активен	включает Upload Protection
Cryptographic Filename Entropy	заменяет/обфусцирует имена файлов
Max Upload Size	жесткий лимит размера Upload
Extension Whitelist	разрешает только определенные расширения

Проверить совместимость: WordPress Editors, Backup Plugins и Business Workflows могут требовать форматы файлов, отсутствующие в слишком узком whitelist.

14. GHOST TRAP - Honeypots

UI позволяет создавать до 50 ловушек, задавать список полиморфных расширений и использовать несколько профилей генерации имен (Mixed, System Fakes, Backup Fakes, Random Hashes). Согласно UI, при сохранении существующая сеть ловушек создается заново.

Эксплуатация: внешние авторизованные Scanner'ы необходимо учитывать и при необходимости разрешать заранее. Любое срабатывание сначала является сигналом; для оценки Incident следует дополнительно использовать контекст Cerberus/XDR.

15. NEMESIS - Deception Grid

Nemesis реализует ограниченный Deception: Tarpits/Delays, Fake Responses и криптографические Canaries. Архитектура прямо указывает, что Hack-Back не применяется. Задержки должны оставаться ограниченными, чтобы намеренно не исчерпывать PHP Worker'ы.

16. STYX - Zero-Trust Egress

Styx перехватывает исходящие WordPress HTTP Calls (`pre_http_request`) и применяет Domain-/Host Whitelist. Это защищает от нежелательной телеметрии и Exfiltration, но способно блокировать легитимные API, Update Servers, платежные сервисы и Webhooks.

Ввод в эксплуатацию: сначала инвентаризировать реально необходимые внешние цели. После активации особенно тщательно протестировать Updates, License Servers, Mail-/SMS Providers, Captcha, платежные сервисы, AI API и собственные Webhooks.

17. CHRONOS - Фоновая проверка целостности

Chronos оркестрирует запланированные Scan'ы. Dashboard позволяет настроить, среди прочего, интервал, получателя, тему и текст предупреждающих E-Mail. Слишком короткий интервал на больших установках может создать ненужную I/O-нагрузку; слишком длинный увеличивает время до обнаружения проблемы.

18. Integrity Monitor и Scanner

Раздел Integrity управляет сравнением Baseline/Hash и показывает новые, измененные, удаленные и классифицированные как Malware файлы. Baseline следует обновлять/approve только тогда, когда изменение легитимно и полностью объяснено.

Правило для Incident: неожиданную модификацию нельзя сразу принимать как новую Baseline. Сначала необходимо проверить источник, Plugin-/Core Update, временную метку, Diff и Malware Finding.

19. KEY VAULT - Secrets

Key Vault хранит API Keys/Secrets под уникальными Identifier'ами. UI указывает AES-256-GCM, AAD Binding и sealed Assets. После сохранения Plaintext Secrets не следует дополнительно дублировать в wp_options, заметках или исходном коде.

Удаление следует считать необратимой операцией. Перед этим необходимо проверить, какие модули используют соответствующий Identifier.

20. THRONEGUARD - Privilege Separation

ThroneGuard отделяет GeDefense Master от обычных администраторов. Он может отнимать отдельные Administrator Capabilities и применять Backend-/REST Gate на основе Superkey с ограниченной по времени Session.

Критические настройки

- **Ограничить роль Administrator:** отнимает выбранные Capabilities у обычных Admin'ов.
- **Заблокировать Master Backend & REST API:** требуется проверка Superkey.
- **Superkey:** согласно UI минимум 12 символов; при изменении текущий Key используется для подтверждения.
- **Restricted Capabilities:** явная матрица отнимаемых прав.

Риск блокировки доступа: перед активацией должно существовать как минимум одно корректно настроенное Master Account и надежно сохраненный Superkey. Изменения manage_options, прав Plugin/Theme/User способны фундаментально изменить административные процессы.

21. LOGINPAGER - локальный Login Interface

LoginPager локально изменяет интерфейс входа WordPress. Настраиваются Background Color, Accent Color, Background Image, Logo, Title и Subtitle. Модуль в первую очередь отвечает за UI/Branding Hardening и должен рассматриваться отдельно от логики доступа Hades/Zeus.

22. ORACLE и Security Center

Oracle выполняет многовекторный Security/Posture Audit и показывает Checks, Status и результаты анализа. **Security Center** агрегирует Assurance, проверки целостности, Trust Architecture, профили прав и Application Confinement Health.

Audit Finding является диагностическим сигналом, а не автоматическим доказательством успешной атаки. Findings следует коррелировать с Logs, XDR Events и целостностью файлов.

23. GORGON - Mesh Sync

Gorgon синхронизирует индикаторы угроз между узлами GeDefense. Dashboard управляет Nexus URL, API Key, Nodes, Ping и Sync. Любое Mesh-соединение расширяет границу доверия; следует использовать только контролируемые HTTPS-цели и Secrets, защищенные Vault.

24. Secure Downloads

Download Manager создает приватные копии с проверкой Hash и временные Download Paths. Администраторы должны относиться к публичным ID/Links как к временным разрешениям и не публиковать их на постоянной основе.

Часть III - Эксплуатация, обслуживание и диагностика

25. Проверка после изменений

После каждого существенного изменения Policy необходимо как минимум:

1. открыть Frontend как анонимный пользователь;
2. заново выполнить Administrator Login;
3. сохранить страницу/запись;
4. протестировать Media Upload;
5. протестировать REST/AJAX-функции важного плагина;
6. протестировать внешний API Call/Webhook, если активен Styx;
7. проверить Security Center, XDR и Logs на новые блокировки;
8. при Titan CSP/Browser Policy проверить Browser Console и Violation View.

26. Типичные неисправности

Администратор не может войти

Проверять по порядку: Hades URL/Secret, Zeus Login Admission/Custom Route, ThroneGuard Superkey, AEGIS/Cerberus Ban и Admin IP Whitelist. Не следует отключать несколько защитных модулей одновременно; причину нужно изолировать послойно.

REST или AJAX перестает работать

Проверить Titan REST Rules, AEGIS Strict, Prometheus Score, Zeus Route Contracts и Admin IP разрешения. Dashboard прямо указывает отсутствие AEGIS-/Prometheus whitelist как возможную причину проблем REST/Nonce/Update.

Не работает внешний API

Проверить Styx Whitelist и Titan `connect -src/Origin Policy`. Для AI дополнительно проверить Vault Key и диагностику Groq.

Upload отклоняется

Проверить Airlock Extension Whitelist, Max Upload, Magic Byte/Scanner Finding и Quarantine.

Frontend теряет Styles/Scripts

Проверить Titan CSP, Observed Origins, Candidate/Report-Only и Enforce State. Не следует сразу задавать глобальные Wildcards; необходимые Origins нужно добавлять точно.

27. Основные принципы Backup и Recovery

- сохранять резервную копию базы данных и файлов плагина перед изменением Policy или версии;
- хранить данные доступа Hades/Zeus/ThroneGuard отдельно от сервера;
- никогда не перезаписывать Integrity Baseline без проверки после Incident;
- экспортированные Server Rules версионировать и проверять их синтаксис перед перезапуском Webserver;
- при Rollback сначала определить внешний слой блокировки: Zeus/Cerberus имеют приоритет перед Policy уровня WordPress.

28. Защита данных и телеметрия

Архитектура содержит компоненты Redaction и псевдонимизированного Export. Тем не менее оператор должен определить, какие Security Logs и как долго хранятся, а также кто имеет к ним доступ. XDR Retention и Export Data должны входить в собственную концепцию защиты данных и Logging.

Часть IV - Полный технический план

Следующая часть включает предоставленную архитектурную карту в качестве технического справочника. Код организован не только по каталогам, но и по фазам загрузки, модулям, Dashboard Views, API, потокам данных, стилям, базам данных и зависимостям. Все пути указаны относительно корневого каталога предоставленного плагина.

Обзор системы

GeDefense WP - Open Core (VisionGaia\GeDefense) - автономная система веб-безопасности и контроля целостности для WordPress, реализованная как полноценный стандартный WordPress Plugin (wp-content/plugins/gedefense-wp-core/).

Система следует строгой **Multi-Phase Ignition Pipeline**:

1. Phase 1: Perimeter Lockdown (Pre-Flight Kernel)

- Запускается синхронно при загрузке основного файла плагина `gedefense-wp.php` в рамках WordPress Plugin Loading Cycle.
- Инициализирует XDR Event Fabric, монтирование File Download Manager и I18n Matrix.
- Cerberus (Priority 0)**: O(1) Memory Lookup для заблокированных IP через APCu/Object Cache. При совпадении - немедленное завершение HTTP 403 менее чем за 0.1 ms.
- AEGIS (Priority 1)**: Deep Packet Inspection WAF для нормализации и блокировки SQLi, XSS, RCE, LFI и атак на десериализацию.
- Striking Queue (Priority 2-N)**: Zeus (6G Pre-Boot WAF), Prometheus (Behavioral Profiling), Nemesis (Asymmetric Deception), Morpheus (RASP Hypervisor) и Gorgon (Telemetry Mesh).

2. Phase 2: Invariant & Hardening Subsystems

- Подключается через стандартный Hook `plugins_loaded` (Priority 10), либо выполняет Phase 2 немедленно, если `plugins_loaded` уже был вызван.
- Запускает Compatibility Manager, Titan (Browser & Surface Confinement), Hades (Admin Cloaking), Airlock (Upload Gateway), Ghost Trap (Honeypots), Chronos (Background Daemon), Styx (Outbound Zero-Trust Firewall), ThroneGuard (Admin Privilege Isolation), LoginPager, Key Vault, Integration Bus и динамический Add-On Registry.
- В административном контексте (`is_admin()`) инициализирует **Dashboard Subsystem** с Glassmorphic UI, AJAX Controllers и Export Pipelines.
- Использует стандартные WordPress Lifecycle Hooks: `register_activation_hook()` для автоматического создания таблиц (`VIS_Schema::enforce()`) и `register_deactivation_hook()` для очистки Cron Schedules и Flush Rewrite Rules.

Ключевые архитектурные инварианты:

- Zero External Dependencies**: без Composer Packages и без CDN (Google Fonts, unpkg, jsdelivr).
- Fail-Closed**: критические Core Subsystems при обнаружении манипуляции завершают работу с HTTP 503 вместо перехода в небезопасный Bypass.
- VGT DIAMANT SUPREME Doctrine**: строгая изоляция Markup, Stylesheets и JavaScript. Zero Inline Styles, Zero Inline Scripts, Zero Inline Event Handlers, Zero DOM String Sinks (`innerHTML`).

1. ГЛОБАЛЬНОЕ ДЕРЕВО АРХИТЕКТУРЫ

GEDEFENSE WP - OPEN CORE (STANDARD WORDPRESS PLUGIN)

— Entrypoints & Bootstrapping	
— <code>gedefense-wp.php</code>	(Стандартная точка входа Plugin, Lifecycle Hooks, константы)
— <code>class-vis-bootstrapper.php</code>	(Master Kernel Orchestrator: запуск Phase 1 & 2)
— <code>class-vis-schema.php</code>	(DB Schema Engine: 10 таблиц, dbDelta, Vault Enforce)
— <code>class-vis-vault.php</code>	(Hardware/Software KMS: Libsodium secretbox & OpenSSL AES-256-GCM)
— Core Infrastructure (includes/core/)	
— <code>class-namespace-compatibility.php</code>	(Центральное ABI и алиасы Legacy Symbols)
— <code>class-vis-security.php</code>	(Zero-Trust Utilities: <code>pinned_https</code> , <code>client_ip</code> , <code>jailed_path</code>)
— <code>class-vis-module-integrity.php</code>	(Криптографический SHA-256 Merkle Tree Invariant Checker)
— <code>class-vis-module-registry.php</code>	(Динамический Open-Core Add-On Hub для VLP, Builder & SEO)
— <code>class-vis-i18n.php</code>	(Zero-Overhead мультязычная матрица DE / EN)
— <code>class-vis-event-bus.php</code>	(Стандартизированный Security Event Emitter & Legacy Logging ABI)
— <code>class-vis-integration-bus.php</code>	(Маршрутизация событий между Sensors и XDR)
— <code>class-vis-security-health.php</code>	(Host Environment Health & PHP / Server Invariant Checker)
— <code>class-vis-security-center.php</code>	(Центральный Self-Test & Diagnostic Reporter Cockpit)
— <code>class-vis-ai-gateway.php</code>	(Изолированный Groq LLM API Transport Adapter с Memory Bounds)
— <code>class-vis-trinity-grid.php</code>	(Interlock Aggregator для Cerberus, Aegis, Prometheus, Nemesis)

Layer 0 - 7 Defense Modules (includes/modules/)	
cerberus/	(L0 Perimeter Drop, O(1) Memory Lookup, OS Rule Exporter)
zeus/	(Pre-Boot 6G WAF, Heuristic Sanitizer & Policy Compiler)
aegis/	(Deep Packet Inspection WAF, Recursive Token Normalizer)
prometheus/	(Когнитивный Behavioral Profiler, /24 Subnet Scoring, Webshell
Heuristics)	
nemesis/	(Asymmetric Deception Grid, Tarpits, Decoy Routing, Frontend
Canaries)	
morpheus/	(Runtime Application Self-Protection / RASP Stack Hypervisor)
gorgon/	(Зашифованная Node Synchronization, Peer-to-Peer Telemetry Mesh)
hades/	(Admin Cloaking, Path Concealment & Authentic 404 Mimicry)
airlock/	(Ingress Upload Streaming Guard, Magic Byte & Polyglot Scanner)
trap/	(Ghost Trap: динамические Decoy Files и немедленные блокировки)
styx/	(Zero-Trust Egress Whitelist Guard, защита от DNS Rebinding)
titan/	(Surface Policy Compiler, CSP, COOP/COEP, WP Hardening)
throneguard/	(Master/Admin Privilege Separation & Superkey Session Gate)
loginpager/	(Zero-Trust локальный Login Interface & Branded Gateway)
chronos/	(Автономный Background Scheduler & File Integrity Daemon)
downloads/	(Secure Download Manager с приватными SHA-256-проверенными копиями)
vault/	(Key Vault с AES-256-GCM & AAD Salt Binding)
oracle/	(12-vector Host & Vulnerability Audit Engine)
filesystem/	(CHMOD Permission & Filesystem Integrity Auditor)
kernel/	(Pre-Boot WAF Diagnostics & Low-Level Sentinel)
TRINITY XDR Subsystem (includes/xdr/)	
class-xdr-event-fabric.php	(Центральный XDR Event Bus & Hook Ingestion)
class-xdr-event.php	(Типизированная неизменяемая XDR Security Event Model)
class-xdr-event-repository.php	(Высокопроизводительное хранилище XDR Events с дедупликацией)
class-xdr-incident-engine.php	(Causal Incident Clustering & Attack Story Generator)
class-xdr-policy-engine.php	(Детерминированная Rule-Based оценка Incidents)
class-xdr-response-engine.php	(Автономные обратимые защитные меры & Rollbacks)
class-xdr-evidence-store.php	(Криптографическая Merkle Evidence Chain для Incidents)
class-xdr-redactor.php	(GDPR-совместимое bounded masking чувствительных параметров)
class-xdr-request-context.php	(Детерминированная привязка Request & Correlation UUID)
Continuous Scanner & Malware Engine (includes/scanner/)	
class-vis-scanner-engine.php	(Пагинированный, изолированный по путям Merkle Tree Scanner)
class-vis-malware-engine.php	(Zero-Dependency Lexical Scanner для Airlock & Chronos)
detectors/	(Специализированные Detectors для PHP, SVG, Archives, Polyglots)
storage/class-vis-quarantine-store.php	(Изолированный приватный Quarantine Vault для угроз)
value/	(Типизированные Value Objects: Context, Budget, Finding, Verdict)
Command Dashboard UI (includes/dashboard/)	
class-vis-dashboard-core.php	(Регистрация Menu: vgt-suite, vgt-throneguard, vgt-loginpager)
class-vis-dashboard-view.php	(Master Glassmorphic Renderer & SVG Topbar Router)
class-vis-dashboard-settings.php	(Whitelist Validation & Module Scoping конфигурации)
class-vis-dashboard-assets.php	(Детерминированный условный Enqueue Engine для CSS/JS)
class-vis-dashboard-ajax.php	(Центральный AJAX Hub: Scanner, Zeus, Unban, Explorer, Add-Ons)
class-vis-sentinel-export.php	(Псевдонимизированный Zero-Dependency JSON Telemetry Exporter)
views/	(27 модульных Dashboard Pages + Sidebar + Sub-Views)
Dynamic Add-On Subsystems (Open-Core Module Hub)	
VLP/	(Vision Legal Pro: Shadow-Net, GDPR Proxy, Lingua AI)
VisionGaiaSEO/	(VisionGaia SEO Architect: Schema, Redirects, Geo-Engine)
builder/	(VisionGaia PageBuilder: Elementor Migrator, Copilot)
Bridge/	(Universal Bridge Adapters, Contracts & Service Providers)
compatibility/	(Third-Party WAF & Builder Bridges: AIOS, Elementor, Divi)
Assets (assets/)	
css/	(vis-dashboard.css, vis-dashboard-modern.css, vis-titan.css, etc.)
js/	(vis-dashboard.js, vis-scanner-client.js, vis-security-center.js,
etc.)	
Cryptographic Trust Anchors (integrity/)	
module-manifest.json	(SHA-256 Merkle Digest Matrix всех 27 Core Components)
Test & Regression Suite (scripts/)	
28 специализированных PHP test scripts	(Liveness, WAF, XDR, Titan, Morpheus, Namespace, Manifest)

2. ПРИВЯЗКА ФАЙЛОВ К АРХИТЕКТУРЕ

2.1 Bootstrapping & Kernel Entry

- `gedefense-wp.php`: основной Standard Plugin Loader; проверяет Double-Load Guards, объявляет глобальные константы (`VIS_VERSION`, `VIS_MANIFEST_DIGEST`, `VIS_PRODUCT_NAME`, `VIS_PATH`, `VIS_URL`, `VIS_VAULT_DIR`), регистрирует Lifecycle Hooks (`register_activation_hook`, `register_deactivation_hook`), принудительно запускает Phase 1 Pre-Flight и подключает Phase 2 к `plugins_loaded`.
- `class-vis-bootstrapper.php`: Core Class `VIS_Bootstrapper`; управляет `engage_phase_1()` (Sovereign Temp Dir, XDR Boot, l18n, Cerberus, Trinity Grid, Aegis, Module Queue) и `engage_phase_2()` (Titan, Hades, Airlock, Chronos, Styx, ThroneGuard, Dashboard).
- `class-vis-schema.php`: Core Class `VIS_Schema`; объявляет определения таблиц (`vis_apex_bans`, `vis_omega_logs`, `vis_oracle_patterns`, `vis_rate_limits`, `vis_xdr_events`, `vis_xdr_incidents`,

`vis_xdr_incident_events`, `vis_xdr_responses`, `vis_xdr_evidence`, `vis_secure_downloads`) и принудительно устанавливает права файлов Vault (0700/0600).

- `class-vis-vault.php`: Core Class `VIS_Vault`; реализует `Libsodium sodium_crypto_secretbox` с Fallback на `OpenSSL aes-256-gcm` и HKDF-вывод ключа из `Salt Material`.

2.2 Core Shared Infrastructure (includes/core/)

- `includes/core/class-namespace-compatibility.php`: `NamespaceCompatibility`; управляет миграциями и Autoloading между каноническим `Namespace VisionGaia\GeDefense` и старыми глобальными символами (`VIS_*`).
- `includes/core/class-vis-security.php`: `VIS_Security`; базовые Security Primitives (`pinned_https_get` с защитой от DNS Rebinding, `client_ip` с Cloudflare Validation, `jailed_path`, `timing_safe_equals`).
- `includes/core/class-vis-module-integrity.php`: `VIS_Module_Integrity`; проверяет Live Hashes файлов по `integrity/module-manifest.json` и вычисляет Merkle Root Hash.
- `includes/core/class-vis-module-registry.php`: `VIS_Module_Registry`; динамически управляет Add-On Modules (VLP, Builder, SEO), проверяет Activation State и Paths.
- `includes/core/class-vis-il18n.php`: `VIS_I18n`; Localization Engine со встроенным словарем DE/EN без Gettext Overhead.
- `includes/core/class-vis-event-bus.php`: `VIS_Event_Bus`; стандартизированный Event Emitter для Security Events; сохраняет события в `wp_vis_omega_logs` и передает их в TRINITY XDR.
- `includes/core/class-vis-integration-bus.php`: `VIS_Integration_Bus`; Cross-Module Message Bus для синхронного уведомления сенсоров об атаках.
- `includes/core/class-vis-security-health.php`: `VIS_Security_Health`; проверка инвариантов Host System (OPcache, OpenSSL, Libsodium, CHMOD, Memory Limits).
- `includes/core/class-vis-security-center.php`: `VIS_Security_Center`; Diagnostic Reporter и Aggregator системного состояния, Integrity Checks и Logs.
- `includes/core/class-vis-ai-gateway.php`: `VIS_AI_Gateway`; абстракционный слой для AI Requests к Groq LLM со строгими Memory Bounds.
- `includes/core/class-vis-trinity-grid.php`: `VIS_Trinity_Grid`; Priming/Interlock механизм для 4 основных компонентов (Cerberus, Aegis, Prometheus, Nemesis).

2.3 Layer 0 - 7 Defense Modules (includes/modules/)

- **Cerberus:**
 - `includes/modules/cerberus/class-vis-cerberus.php`: `VIS_Cerberus`; O(1) Memory Drop, APCu IP Cache, экспорт Nginx/Apache/nftables Ban Lists.
- **Zeus:**
 - `includes/modules/zeus/class-vis-zeus.php`: `VIS_Zeus`; основная WAF Class.
 - `includes/modules/zeus/src/class-zeus-compiler.php`: создает атомарные RegEx DFA.
 - `includes/modules/zeus/src/class-zeus-shield.php`: Pre-Boot filtering против 6G/Author Scan.
 - `includes/modules/zeus/src/class-zeus-blackbox.php`: Telemetry Buffer для Block Decisions.
 - `includes/modules/zeus/src/class-zeus-admission.php`: Token-Based WAF Bypass Admission.
 - `includes/modules/zeus/src/class-zeus-benchmark.php`: Performance и Latency Benchmarking WAF.
 - `includes/modules/zeus/src/class-zeus-budget.php`: контроллер CPU/Time Budget.
 - `includes/modules/zeus/src/class-zeus-contracts.php`: определения Interface для WAF Rules.
 - `includes/modules/zeus/src/class-zeus-edge.php`: Edge Rule Compiler для Reverse Proxies.
 - `includes/modules/zeus/src/class-zeus-env.php`: проверки Environment и Host.
 - `includes/modules/zeus/src/class-zeus-envelope.php`: инкапсуляция Request Envelope.
 - `includes/modules/zeus/src/class-zeus-learning.php`: эвристическое обучение на легитимных запросах.
 - `includes/modules/zeus/src/class-zeus-policy-manager.php`: управление активными Policies.
 - `includes/modules/zeus/src/class-zeus-vault-resolver.php`: разрешение ключей для WAF Tokens.
 - `includes/modules/zeus/src/class-zeus-xdr-bridge.php`: Bridge для отправки событий в TRINITY XDR.
- **Aegis:**
 - `includes/modules/aegis/class-vis-aegis.php`: `VIS_Aegis`; Deep Packet Inspection Engine (DPI). Рекурсивная Token Normalization для GET, POST, JSON, Multi-Part, Headers.
 - `includes/modules/aegis/class-vis-aegis-oracle.php`: `VIS_Aegis_Oracle`; эвристический анализатор Attack Patterns.
- **Prometheus:**
 - `includes/modules/prometheus/class-vis-prometheus.php`: `VIS_Prometheus`; Behavioral Profiler, IP/Subnet Scoring, Decay Algorithm, PHP Webshell Scanner.
- **Nemesis:**
 - `includes/modules/nemesis/class-vis-nemesis.php`: `VIS_Nemesis`; Asymmetric Deception, Tarpits, Decoy Routing, Frontend HMAC Canaries.
- **Morpheus:**
 - `includes/modules/morpheus/class-vis-morpheus.php`: `Vis_Morpheus`; точка входа RASP.

- `includes/modules/morpheus/src/class-morpheus-hypervisor.php`: Hypervisor для Live Stacktraces.
- `includes/modules/morpheus/src/class-morpheus-path-jail.php`: изоляция PHP Paths.
- `includes/modules/morpheus/src/class-morpheus-tracer.php`: Execution Tracer для Hook Auditing.
- `includes/modules/morpheus/src/class-morpheus-ui.php`: Dashboard UI Components.
- `includes/modules/morpheus/src/class-morpheus-dashboard.php`: AJAX Handler для RASP Approvals.
- `includes/modules/morpheus/src/class-morpheus-ai.php`: AI-assisted Callstack Analysis.
- `includes/modules/morpheus/src/shields/class-morpheus-shield-db.php`: DML Protection для `wp_users`.
- `includes/modules/morpheus/src/shields/class-morpheus-shield-network.php`: SSRF Block для Cloud Metadata (169.254.169.254).
- `includes/modules/morpheus/src/shields/class-morpheus-shield-state.php`: State Verification.
- **Gorgon:**
 - `includes/modules/gorgon/class-vis-gorgon.php`: `Vis_Gorgon`; оркестрация P2P Mesh Nodes.
 - `includes/modules/gorgon/src/class-gorgon-sync-engine.php`: Synchronization Pipeline.
 - `includes/modules/gorgon/src/class-gorgon-harvester.php`: собирает локальные Ban Lists и Threats.
 - `includes/modules/gorgon/src/class-gorgon-uplink.php`: защищенный HTTPS Uplink к Peer Nodes.
 - `includes/modules/gorgon/src/class-gorgon-config.php`: Configuration Model для Mesh Networks.
 - `includes/modules/gorgon/src/class-vis-gorgon-ajax.php`: специализированный AJAX Controller для Gorgon.
- **Hades:**
 - `includes/modules/hades/class-vis-hades.php`: `VIS_Hades`; скрывание `/wp-admin` и `wp-login.php`, криптографические HMAC Cookie Gates, 404 Mimicry.
- **Airlock:**
 - `includes/modules/airlock/class-vis-airlock.php`: `VIS_Airlock`; Hook в `wp_handle_upload_prefilter`.
 - `includes/modules/airlock/src/class-airlock-scanner.php`: Magic Byte Check и обнаружение Polyglot.
 - `includes/modules/airlock/src/class-airlock-sanitizer.php`: очистка XML/SVG против XSS/XXE.
 - `includes/modules/airlock/src/class-airlock-config.php`: Policies по File Type и Size.
- **Ghost Trap:**
 - `includes/modules/trap/class-vis-ghost-trap.php`: `VIS_Ghost_Trap`; мониторинг Decoy Paths (`.env`, `.git`, `backup.sql`).
 - `includes/modules/trap/src/class-ghost-trap-engine.php`: Response Generator и Ban Trigger.
 - `includes/modules/trap/src/class-ghost-trap-authenticator.php`: Token Check для легитимных Scan'ов.
 - `includes/modules/trap/src/class-ghost-trap-config.php`: конфигурация списка Decoys.
- **Styx:**
 - `includes/modules/styx/class-vis-styx.php`: `VIS_Styx`; подключается к `pre_http_request`. Zero-Trust Egress Whitelist, защита от SSRF и DNS Rebinding.
- **Titan:**
 - `includes/modules/titan/class-vis-titan.php`: `VIS_Titan`; Master Hardening Class.
 - `includes/modules/titan/src/class-titan-surface-resolver.php`: определяет Context (Public, Login, Admin, REST, AJAX).
 - `includes/modules/titan/src/class-titan-policy-compiler.php`: компилирует CSP, Permissions-Policy, COOP, COEP, HSTS.
 - `includes/modules/titan/src/class-titan-policy-store.php`: управляет и версионизирует Security Policies.
 - `includes/modules/titan/src/class-titan-runtime.php`: Header Injection и Conflict Resolution.
 - `includes/modules/titan/src/class-titan-violation-collector.php`: собирает CSP Reports через REST API.
 - `includes/modules/titan/src/class-titan-learning.php`: эвристическое обучение Domain Origins.
 - `includes/modules/titan/src/class-titan-server-rules.php`: экспортирует Nginx/Caddy Header Configurations.
 - `includes/modules/titan/src/class-titan-login-gate.php`: One-Time Token для Login Access.
 - `includes/modules/titan/src/class-titan-sandbox.php`: изолированный Preview активного контента.
 - `includes/modules/titan/src/class-titan-recovery.php`: Emergency Recovery CSP через WP-CLI.
 - `includes/modules/titan/src/class-titan-assurance.php`: проверяет Security State HTTP Headers.
- **ThroneGuard:**
 - `includes/modules/throneguard/class-vis-throne-guard.php`: `VIS_Throne_Guard`; изоляция Master Administrator, удаление опасных Super Admin Rights, Fingerprint-bound Superkey Sessions.
- **LoginPager:**
 - `includes/modules/loginpager/class-vis-loginpager.php`: `VIS_LoginPager`; локальный Hardening и оформление Login Page (`login_enqueue_scripts`, `login_headerurl`).
- **Chronos:**
 - `includes/modules/chronos/class-vis-chronos.php`: `VIS_Chronos`; WP-Cron Autopilot.
 - `includes/modules/chronos/src/class-chronos-engine.php`: выполняет периодические Filesystem Scans.

- `includes/modules/chronos/src/class-chronos-scheduler.php`: управляет Cron Intervals (`vis_hourly_scan_event`).
- `includes/modules/chronos/src/class-chronos-alerter.php`: отправляет Alert E-Mails при нарушении целостности.
- `includes/modules/chronos/src/class-chronos-config.php`: Thresholds и Recipients.
- **Secure Downloads:**
 - `includes/modules/downloads/class-secure-download-manager.php`: `DownloadManager`; приватные неизменяемые копии файлов с SHA-256 Integrity Proof и временными Download Routes.
- **Key Vault:**
 - `includes/modules/vault/class-vis-key-vault.php`: `VIS_Key_Vault`; хранит чувствительные API Keys (например AIOS, Groq, Cloudflare), зашифрованные AES-256-GCM с AAD Binding.
- **Oracle:**
 - `includes/modules/oracle/class-vis-oracle.php`: `VIS_Oracle`; 12-vector Audit Engine для обнаружения ошибок конфигурации и уязвимостей.
- **Filesystem Guard:**
 - `includes/modules/filesystem/class-vis-filesystem-guard.php`: `VIS_Filesystem_Guard`; аудит CHMOD Rights для `wp-config.php`, `.htaccess`, `/uploads/` и Core Directories.
- **Kernel Sentinel:**
 - `includes/modules/kernel/class-vis-kernel-sentinel.php`: `VIS_Kernel_Sentinel`; Low-Level Monitoring PHP Limits и Boot Integrity.

2.4 TRINITY XDR Subsystem (`includes/xdr/`)

- `class-xdr-event-fabric.php`: центральный Ingestion Channel для Security Events всех сенсоров.
- `class-xdr-event.php`: неизменяемое XDR Security Event с Severity, Category и Actor Hash.
- `class-xdr-event-repository.php`: CRUD и Search Operations по `wp_vis_xdr_events`.
- `class-xdr-incident-engine.php`: коррелирует отдельные эффекты в Attack Incidents (`wp_vis_xdr_incidents`).
- `class-xdr-policy-engine.php`: применяет динамические Response Policies к Incidents.
- `class-xdr-response-engine.php`: выполняет автоматические Countermeasures, например IP Ban и Session Kill.
- `class-xdr-evidence-store.php`: audit-safe Merkle Evidence Store в `wp_vis_xdr_evidence`.
- `class-xdr-redactor.php`: удаляет Passwords, Credit Card Data и чувствительные значения из Log Entries.
- `class-xdr-request-context.php`: генерирует и связывает Request UUID и Correlation ID.

2.5 Scanner & Malware Engine (`includes/scanner/`)

- `class-vis-scanner-engine.php`: пагинированный File Scanner для всего Web Root с хранением Baseline в `VIS_VAULT_DIR`.
- `class-vis-malware-engine.php`: Shared Detector Engine для синтаксического и эвристического анализа угроз.
- `detectors/class-vis-php-lexical-detector.php`: Token-based AST Scanner для Backdoors, Obfuscation и `eval()`.
- `detectors/class-vis-polyglot-detector.php`: проверяет Images и Archives на встроенный PHP Code.
- `detectors/class-vis-svg-xml-detector.php`: XML Parser для обнаружения SVG-based XSS и XXE.
- `detectors/class-vis-archive-detector.php`: инспектирует ZIP, TAR и GZ Archives на подозрительное содержимое.
- `detectors/class-vis-path-context-detector.php`: контекстная проверка Path и Filename Patterns.
- `storage/class-vis-quarantine-store.php`: изолирует зараженные файлы в зашифрованном каталоге.
- `contracts/interface-vis-file-detector.php`: стандартизированный Interface для всех File Detectors.
- `value/class-vis-scan-budget.php`, `class-vis-scan-context.php`, `class-vis-scan-finding.php`, `class-vis-scan-verdict.php`: типизированные Data Structures для Scan Runs.

2.6 Command Dashboard (`includes/dashboard/`)

- `class-vis-dashboard-core.php`: регистрация Menu и инициализация Enqueue.
- `class-vis-dashboard-view.php`: Layout Rendering, SVG Navigation и Tab Router.
- `class-vis-dashboard-settings.php`: обрабатывает Form Submissions с Nonce и Type Validation.
- `class-vis-dashboard-assets.php`: управляет условной Enqueue Logic для 27 Views и Global Assets.
- `class-vis-dashboard-ajax.php`: центральная точка для AJAX Actions (`vis_run_scan`, `vis_approve_changes`, etc.).
- `class-vis-sentinel-export.php`: создает audit-safe JSON Exports системной конфигурации и Logs.
- `views/`:
 - `view-overview.php`: Control Center & HUD.
 - `view-thread.php`: XDR Threat Matrix & Event Log.
 - `view-oracle.php`: Oracle System Diagnostics.
 - `view-integrity.php`: Integrity Checker & File Explorer.
 - `view-security_center.php`: консолидированный Security Center (Assurance, System Status, Logs).

- `view-systatus.php`: Host & PHP Infrastructure Diagnostics.
- `view-logs.php`: searchable Logbook.
- `view-trinity.php`: Interlock Matrix четырех защитных Shields.
- `view-zeus.php`: Zeus WAF Cockpit, Compiler & Benchmark.
- `view-aegis.php`: AEGIS WAF Configuration & Ruleset.
- `view-prometheus.php`: Prometheus Heuristics & Subnet Monitor.
- `view-cerberus.php`: IP Ban List & Ban Management.
- `view-airlock.php`: Upload Filter Configuration.
- `view-nemesis.php`: Tarpit & Honeytoken Configuration.
- `view-ghost_trap.php`: Decoy File Configuration.
- `view-hades.php`: Admin Cloaking Paths & Cloaking.
- `view-morpheus.php`: RASP Callstack Inspector & Shields.
- `view-titan.php`: Browser Confinement & CSP Compiler.
- `view-kernel.php`: Kernel Status & Low-Level Parameters.
- `view-styx.php`: Egress Outbound Whitelist.
- `view-chronos.php`: Background Scanner Schedule & Alerts.
- `view-vlp.php`: Data Protection & Shadow-Net Configuration.
- `view-filesystem.php`: Filesystem Permissions.
- `view-vault.php`: Crypto Key Vault.
- `view-throneguard.php`: Master Admin Privilege Management.
- `view-loginpager.php`: Login Surface Designer.
- `view-downloads.php`: Secure Download Files.
- `view-modules.php`: Add-On Package Manager.
- `view-setup_wizard.php`: 7-step Setup Wizard.
- `view-xdr.php`: detailed XDR Incident Situation View.
- `view-gorgon.php`: P2P Mesh Status & Sync Log.
- `view-sidebar.php`: Global Sidebar Component.
- `view-aos.php`: Legacy AIOS Bridge Status.
- `view-myrmidon.php`: Legacy Myrmidon Device Trust.

3. ДОКУМЕНТАЦИЯ МОДУЛЕЙ

Модуль: Cerberus (L0 Perimeter Guard)

Назначение

Обеспечивает бескомпромиссную защиту периметра Layer 0, отбрасывая вредоносные запросы менее чем за 0.1 ms еще до того, как WordPress откроет соединение с базой данных или загрузит Themes/Plugins.

Возможности

- O(1) Memory Lookup заблокированных IP через APCu и Object Cache.
- Немедленное завершение с HTTP 403 Forbidden при минимальном CPU Overhead.
- Динамическая блокировка при превышении порогов Aegis, Zeus, Prometheus или Ghost Trap.
- Автоматический экспорт нативных Webserver/Firewall Ban Rules (`nginx_deny.conf`, `htaccess_deny.conf`, `nftables_drop.map`).

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-cerberus.php
- includes/dashboard/views/cerberus/script.js

Backend:
- includes/modules/cerberus/class-vis-cerberus.php

API:
- wp_ajax_vis_dashboard_unban_ip (in class-vis-dashboard-ajax.php)

Styles:
- includes/dashboard/views/cerberus/style.css

Database:
```

```
- wp_vis_apex_bans

Configuration:
- Option 'vis_config' (Keys: cerberus_enabled, cerberus_autoban)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> cerberus
- **Компоненты:** Ban Table с Pagination, Unban Modal, ручная IP-блокировка, Download Webserver Rules.
- **Styles:** cerberus/style.css
- **APIs:** wp_ajax_vis_dashboard_unban_ip
- **Backend Functions:** VIS_Cerberus::instance()->unban_ip(), VIS_Cerberus::instance()->ban_ip()
- **Данные:** список заблокированных IP, причина, Timestamp, Request URI.
- **Действия пользователя:** вручную заблокировать IP, снять блокировку, скачать OS Firewall Rules.

Зависимости

- Требуется Core: VIS_Security::client_ip()
- Читает/пишет таблицу: wp_vis_apex_bans
- Отправляет Events в: VIS_Event_Bus, TRINITY XDR

Используется в

- VIS_Bootstrapper::engage_phase_1() (Priority 0)
- VIS_Aegis (Ban при тяжелой попытке Exploit)
- VIS_Prometheus (Ban при превышении Threat Score)
- VIS_Ghost_Trap (немедленный Ban при обращении к Decoy Files)
- VIS_Dashboard_Ajax (Unban через Admin UI)

Модуль: Zeus (6G Pre-Boot WAF)

Назначение

Блокирует вредоносные запросы (Malicious Query Strings, Bad Bots, Author Enumeration) на основе скомпилированных детерминированных Regular Expressions (DFA) до выполнения дорогостоящей PHP-логики.

Возможности

- Высокоскоростная проверка Request URI, User Agent и Referer.
- Автоматическая компиляция 6G/7G Signatures в оптимизированные PHP RegEx Trees (Zeus_Compiler).
- Admission Token Generator для авторизованных Scanner'ов.
- Latency Benchmarking и CPU Budgeting.
- Прямая Bridge-интеграция для передачи Block Events в TRINITY XDR.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-zeus.php
- includes/dashboard/views/zeus/script.js

Backend:
- includes/modules/zeus/class-vis-zeus.php
- includes/modules/zeus/src/class-zeus-compiler.php
- includes/modules/zeus/src/class-zeus-shield.php
- includes/modules/zeus/src/class-zeus-blackbox.php
- includes/modules/zeus/src/class-zeus-admission.php
- includes/modules/zeus/src/class-zeus-benchmark.php
- includes/modules/zeus/src/class-zeus-budget.php
- includes/modules/zeus/src/class-zeus-config-repository.php
- includes/modules/zeus/src/class-zeus-contracts.php
- includes/modules/zeus/src/class-zeus-edge.php
- includes/modules/zeus/src/class-zeus-env.php
- includes/modules/zeus/src/class-zeus-envelope.php
- includes/modules/zeus/src/class-zeus-learning.php
- includes/modules/zeus/src/class-zeus-policy-manager.php
- includes/modules/zeus/src/class-zeus-vault-resolver.php
- includes/modules/zeus/src/class-zeus-xdr-bridge.php

API:
- wp_ajax_vis_save_zeus_config
- wp_ajax_vis_zeus_run_benchmark
- wp_ajax_vis_zeus_run_self_test
- wp_ajax_vis_zeus_drain_blackbox
```

```

- wp_ajax_vis_zeus_restore_preset
- wp_ajax_vis_zeus_save_contract
- wp_ajax_vis_zeus_delete_contract
- wp_ajax_vis_zeus_generate_admission_token
- wp_ajax_vis_zeus_rollback_policy

Styles:
- includes/dashboard/views/zeus/style.css

Database:
- Options: 'vis_zeus_config', 'vis_zeus_contracts'

Tests:
- scripts/zeus-env-regression.php
- scripts/zeus-live-waf-regression.php
- scripts/zeus-nextgen-regression.php

```

Интеграция с Dashboard

- **Страницы:** Dashboard -> zeus
- **Компоненты:** Telemetry Tiles, Blackbox Drain Monitor, Benchmark Runner, Admission Token Manager, Policy Compiler Matrix.
- **Styles:** zeus/style.css
- **APIs:** 9 специализированных AJAX Endpoints `vis_zeus_*`.
- **Backend Functions:** `Zeus_Compiler::compile()`, `Zeus_Benchmark::run()`, `Zeus_Blackbox::drain()`
- **Данные:** статистика блокировок, latency в микросекундах, активные Rulesets.
- **Действия пользователя:** ужесточить/ослабить WAF Rules, запустить Benchmark, создать Emergency Admission Token, очистить Blackbox Buffer.

Зависимости

- Требуется Core: `VIS_Security`, `VIS_Vault` (Token Signing)
- Передает данные в: TRINITY XDR через `Zeus_Xdr_Bridge`
- Читает таблицу: `wp_vis_rate_limits`

Используется в

- `VIS_Bootstrapper::engage_phase_1()` (Priority 2)
- `VIS_Trinity_Grid` (State Check)

Модуль: Aegis (Deep Packet Inspection WAF)

Назначение

Рекурсивно анализирует входящие Payloads (GET, POST, JSON, XML, HTTP Headers, Cookies) до глубины 15 уровней на SQL Injection, Cross-Site Scripting (XSS), Remote Code Execution (RCE), Local File Inclusion (LFI) и небезопасную десериализацию.

Возможности

- Двухфазная Pipeline: быстрое DFA Signature Matching, затем рекурсивная Normalization (разрешение SQL Comments, Hex/Unicode Homoglyphs, Quote/Slash Slicing).
- Эвристическая классификация Attack Vectors через `VIS_Aegis_Oracle`.
- Настраиваемый Whitelist для Admin IP и известных User Agents.
- Непосредственный Trigger Cerberus Ban при критических Exploit'ах.

Связанные файлы

```

Frontend / View:
- includes/dashboard/views/view-aegis.php
- assets/js/vis-oracle-diagnostics.js

Backend:
- includes/modules/aegis/class-vis-aegis.php
- includes/modules/aegis/class-vis-aegis-oracle.php

API:
- wp_ajax_vis_oracle_ping (Diagnostics Ping)

Styles:
- includes/dashboard/views/aegis/style.css

Database:
- Options: 'vis_config' (Keys: aegis_enabled, aegis_mode, aegis_whitelist_ips, aegis_whitelist_uas)
- Logs in: 'wp_vis_omega_logs'

Tests:

```

```
- scripts/aegis-regression.php
- scripts/sentinel-threat-benchmark.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `aegis`
- **Компоненты:** Mode Switch (Monitor / Enforce / Aggressive), Whitelist Editor для IP/User Agents, Live Inspection Diagnostics.
- **Styles:** `aegis/style.css`
- **APIs:** `vis_oracle_ping`
- **Backend Functions:** `VIS_Aegis::inspect()`, `VIS_Aegis_Oracle::analyze()`
- **Данные:** Block Metrics, Attack Vectors, Whitelist Configuration.
- **Действия пользователя:** изменить Protection Mode, обновить Whitelist'ы, симулировать Test Attacks против WAF.

Зависимости

- Требуется Core: `VIS_Security::client_ip()`
- Вызывает: `VIS_Cerberus::ban_ip()`, `VIS_Event_Bus::log()`
- Связан с: `VIS_Trinity_Grid`

Используется в

- `VIS_Bootstrapper::engage_phase_1()` (Priority 1)

Модуль: Prometheus (Behavioral Profiler & Heuristics)

Назначение

В реальном времени отслеживает поведение IP-адресов и /24 подсетей, накапливает Threat Scores по временным окнам и противодействует Brute Force и поведенческим аномалиям.

Возможности

- Динамический Scoring на основе частоты Requests, генерации 404 и WAF Hits.
- Математический Score Decay - снижение безвредных Scores со временем.
- Обнаружение вредоносных /24 Subnet Clustering Attacks.
- Встроенный PHP Webshell Signature Scanner для POST Payloads.
- Атомарные механизмы блокировки через MySQL GET_LOCK против Race Conditions.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-prometheus.php
- includes/dashboard/views/prometheus/script.js

Backend:
- includes/modules/prometheus/class-vis-prometheus.php

API:
- Mutations обрабатываются через VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/prometheus/style.css

Database:
- Options: 'vis_config' (Keys: prometheus_enabled, prometheus_threshold, prometheus_whitelist_ips)
- Table: 'wp_vis_omega_logs' (Subsystem Prometheus)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `prometheus`
- **Компоненты:** Anomaly Gauge, Subnet Threat Matrix, Heuristic Threshold Controller.
- **Styles:** `prometheus/style.css`
- **APIs:** Form Mutations через Standard Settings.
- **Backend Functions:** `VIS_Prometheus::evaluate_request()`
- **Данные:** текущая Risk Assessment Host, заблокированные Subnets, Behavioral Scores.
- **Действия пользователя:** настраивать Sensitivity Scales, поддерживать IP Whitelist.

Зависимости

- Требуется Core: `VIS_Security::client_ip()`
- Вызывает `VIS_Cerberus::ban_ip()` при превышении Threshold.

Используется в

- `VIS_Bootstrapper::engage_phase_1()` (Priority 3)

Модуль: Nemesis (Asymmetric Cyber Deception Grid)

Назначение

Создает асимметричные издержки для атакующего с помощью bounded Tarpits (Micro Delays), поддельных API Responses, измененных Status Codes и внедрения криптографических Frontend Tracking Canaries.

Возможности

- Bounded Delay Tarpits без исчерпания PHP Worker Threads.
- Deception Responses (Fake Exploit Success) для атакующего с одновременным внутренним Alert.
- Инъекция Honeytokens (HMAC-SHA256 Canary Tokens) в HTML Source.
- Строго оборонительный подход: без Hack-Back Payloads и Response Bombs; полностью совместим с RFC и правовыми ограничениями.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-nemesis.php
- includes/dashboard/views/nemesis/script.js

Backend:
- includes/modules/nemesis/class-vis-nemesis.php

API:
- Settings через VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/nemesis/style.css

Database:
- Options: 'vis_config' (Keys: nemesis_enabled, nemesis_tarpit_delay, nemesis_fake_responses)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `nemesis`
- **Компоненты:** Deception Grid Overview, Tarpit Latency Configurator, Canary Token Generator.
- **Styles:** `nemesis/style.css`
- **APIs:** Form Mutations через Settings.
- **Backend Functions:** `VIS_Nemesis::deploy_deception()`, `VIS_Nemesis::trap()`
- **Данные:** количество введенных в заблуждение Scanner'ов, активные Honeytokens.
- **Действия пользователя:** изменить Tarpit Delay, активировать Fake Routes.

Зависимости

- Требуется Core: `VIS_Security`, `VIS_Vault`
- Сообщает Incidents в: `TRINITY XDR`, `VIS_Cerberus`

Используется в

- `VIS_Bootstrapper::engage_phase_1()` (Priority 4)

Модуль: Morpheus (RASP Hypervisor & Execution Sandbox)

Назначение

Runtime Application Self-Protection (RASP): динамически контролирует PHP Callstack во время выполнения, защищает базу данных от несанкционированных изменений таблицы `wp_users` и предотвращает Server-Side Request Forgery (SSRF) к Cloud Metadata.

Возможности

- Live Stacktrace Inspection для проверки вызывающих Plugins и Scripts.
- Database Shield: перехватывает DML Operations (UPDATE, DELETE, INSERT) для `wp_users` и `wp_usermeta` и требует криптографические Authorization Tokens.
- Network Shield: блокирует исходящие Requests к Cloud Metadata (`169.254.169.254`) и приватным IP Ranges.

- Filesystem Jail: запрещает неразрешенные File Operations вне Webroot.
- AI-assisted обнаружение Callstack Anomalies через Groq AI Gateway.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-morpheus.php
- includes/dashboard/views/morpheus/script.js

Backend:
- includes/modules/morpheus/class-vis-morpheus.php
- includes/modules/morpheus/src/class-morpheus-hypervisor.php
- includes/modules/morpheus/src/class-morpheus-path-jail.php
- includes/modules/morpheus/src/class-morpheus-tracer.php
- includes/modules/morpheus/src/class-morpheus-ui.php
- includes/modules/morpheus/src/class-morpheus-dashboard.php
- includes/modules/morpheus/src/class-morpheus-ai.php
- includes/modules/morpheus/src/shields/class-morpheus-shield-db.php
- includes/modules/morpheus/src/shields/class-morpheus-shield-network.php
- includes/modules/morpheus/src/shields/class-morpheus-shield-state.php

API:
- wp_ajax_vgt_morpheus_trigger_ai
- wp_ajax_vgt_morpheus_reject_ai
- wp_ajax_vgt_morpheus_approve_ai
- wp_ajax_vgt_morpheus_delete_matrix
- wp_ajax_vgt_morpheus_toggle_strict

Styles:
- includes/dashboard/views/morpheus/style.css

Database:
- Options: 'vgt_morpheus_audit_matrix', 'vis_config'

Tests:
- scripts/morpheus-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `morpheus`
- **Компоненты:** RASP Hypervisor Status, Hook Audit Matrix, DML Block Monitor, AI Stack Analysis.
- **Styles:** `morpheus/style.css`
- **APIs:** 5 специализированных AJAX Endpoints для RASP Approval и AI Trigger.
- **Backend Functions:** `Vis_Morpheus::engage()`, `Morpheus_Shield_DB::intercept()`
- **Данные:** проаудированные PHP Callstacks, обнаруженные Injection Attempts.
- **Действия пользователя:** разрешать/блокировать Plugins в Audit Matrix, переключать Strict Mode.

Зависимости

- Требуется Core: `VIS_AI_Gateway`, `VIS_Security`
- Интегрируется в: `$wpdb Query Filters`

Используется в

- `VIS_Bootstrapper::engage_phase_1()` (Priority 5)

Модуль: Gorgon (Encrypted Mesh Sync & Nexus Uplink)

Назначение

Обеспечивает децентрализованную криптографически защищенную Node-to-Node синхронизацию IP Ban Lists и Threat Intelligence между несколькими экземплярами GeDefense.

Возможности

- P2P Mesh Uplink с проверкой HMAC-SHA256 Signature.
- Автоматический Import/Export Threat Indicators (IOC).
- Изолированный AJAX Controller для предотвращения Header Collisions.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-gorgon.php
- includes/dashboard/views/gorgon/script.js

Backend:
- includes/modules/gorgon/class-vis-gorgon.php
- includes/modules/gorgon/src/class-gorgon-sync-engine.php
- includes/modules/gorgon/src/class-gorgon-harvester.php
- includes/modules/gorgon/src/class-gorgon-uplink.php
```

```
- includes/modules/gorgon/src/class-gorgon-config.php
- includes/modules/gorgon/src/class-vis-gorgon-ajax.php

API:
- wp_ajax_vgt_gorgon_toggle
- wp_ajax_vgt_gorgon_update_config
- wp_ajax_vgt_gorgon_ping_nexus
- wp_ajax_vgt_gorgon_sync
- wp_ajax_vgt_gorgon_add_node
- wp_ajax_vgt_gorgon_remove_node

Styles:
- includes/dashboard/views/gorgon/style.css

Database:
- Options: 'vis_config' (Keys: gorgon_enabled, gorgon_nexus_url, gorgon_api_key)

Tests:
- scripts/integration-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `gorgon` (доступна через прямую ссылку Tab)
- **Компоненты:** Nexus Node Status, Sync History, Peer Node Management.
- **Styles:** `gorgon/style.css`
- **APIs:** 6 специализированных AJAX Routes `vgt_gorgon_*`.
- **Backend Functions:** `Gorgon_Sync_Engine::sync()`, `Gorgon_Uplink::ping()`
- **Данные:** подключенные Nodes, Latency, обмененные IP Bans.
- **Действия пользователя:** добавлять/удалять Nodes, вручную запускать Synchronization.

ЗАВИСИМОСТИ

- Требуется Core: `VIS_Security::pinned_https_get()`, `VIS_Vault`

Используется в

- `VIS_Bootstrapper::engage_phase_1()` (Priority 6)

Модуль: Hades (Admin Cloaking & 404 Mimicry)

Назначение

Маскирует административные пути (`/wp-admin`, `wp-login.php`), блокирует неавторизованные прямые обращения с правдоподобными Webserver 404 Pages и разрешает Admin Access только через секретные GET Parameters или HMAC-protected Cookies.

Возможности

- Полное скрывание стандартного Login URL.
- Timing-safe HMAC-SHA256 Gate Cookie Check.
- Автоматический Fallback на реальные Apache/Nginx 404 Status Codes при неавторизованном доступе.
- Компиляция нативных Webserver Rewrite Rules для Nginx и Apache.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-hades.php

Backend:
- includes/modules/hades/class-vis-hades.php

API:
- Mutations via VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/hades/style.css

Database:
- Options: 'vis_config' (Keys: hades_enabled, hades_secret_param, hades_secret_value, hades_stealth_mode)

Tests:
- scripts/hades-shadow-net-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `hades`
- **Компоненты:** Secret URL Generator, Bypass Link Display, Nginx/Apache Rewrite Rule Box.
- **Styles:** `hades/style.css`
- **APIs:** сохранение Configuration.

- **Backend Functions:** `VIS_Hades::protect_admin()`, `VIS_Hades::get_nginx_rules()`
- **Данные:** текущий Secret Key, авторизованные Admin Sessions.
- **Действия пользователя:** настроить Secret Parameter, копировать Rewrite Rules.

Зависимости

- Требуется Core: `VIS_Security::validate_hades_gate()`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`

Модуль: Airlock (Ingress Polyglot & Upload Sanitizer)

Назначение

В реальном времени анализирует File Upload через `wp_handle_upload_prefilter`, обнаруживает Polyglot Files (например PHP Code внутри Image Header) и нейтрализует SVG XSS/XXE Payloads.

Возможности

- Проверка реальных MIME Types через Magic Bytes, а не по File Extension.
- Полный XML Parsing SVG-файлов с удалением `<script>`, `onload` и External Entities (XXE).
- Обнаружение встроенных PHP Tags (`<?php`, `<?=>`, `<script language="php">`) внутри Image Files.
- Автоматическое помещение подозрительных Uploads в изолированный Quarantine Vault.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-airlock.php

Backend:
- includes/modules/airlock/class-vis-airlock.php
- includes/modules/airlock/src/class-airlock-scanner.php
- includes/modules/airlock/src/class-airlock-sanitizer.php
- includes/modules/airlock/src/class-airlock-config.php

API:
- Filter: 'wp_handle_upload_prefilter'

Styles:
- includes/dashboard/views/airlock/style.css

Database:
- Options: 'vis_config' (Keys: airlock_enabled, airlock_quarantine_enabled, airlock_max_file_size)

Tests:
- scripts/malware-scanner-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `airlock`
- **Компоненты:** Upload Policy Table, SVG Protection Status, Quarantine Settings.
- **Styles:** `airlock/style.css`
- **APIs:** Standard Settings Mutation.
- **Backend Functions:** `VIS_Airlock_Scanner::scan_file()`, `VIS_Airlock_Sanitizer::sanitize_svg()`
- **Данные:** заблокированные Uploads, Quarantine Status.
- **Действия пользователя:** определить разрешенные File Types, включить Quarantine.

Зависимости

- Использует Scanner: `VIS_Malware_Engine`, `VIS_Quarantine_Store`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`
- `VIS_Titan` (проверка Upload Certificates)

Модуль: Ghost Trap (Dynamic Decoy Honeypots)

Назначение

Размещает на сайте динамические Decoy Files и Routes (`.env`, `.git/config`, `wp-config.php.bak`, `phpinfo.php`), которые легитимный пользователь никогда не запрашивает, и немедленно блокирует Scanner при обращении к ним.

Возможности

- Перехватывает Requests к особо чувствительным File Names.
- Немедленно вызывает постоянный Cerberus Ban для IP атакующего.
- Возвращает правдоподобный, но безопасный Fake Content.
- Не создает нагрузку на реальные файлы диска: маршрутизация полностью виртуальная.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-ghost_trap.php

Backend:
- includes/modules/trap/class-vis-ghost-trap.php
- includes/modules/trap/src/class-ghost-trap-engine.php
- includes/modules/trap/src/class-ghost-trap-authenticator.php
- includes/modules/trap/src/class-ghost-trap-config.php

API:
- Mutations via VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/ghost_trap/style.css

Database:
- Options: 'vis_config' (Keys: ghost_trap_enabled, ghost_trap_exts)

Tests:
- scripts/ghost-trap-authentication-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `ghost_trap`
- **Компоненты:** Decoy Matrix, Hit Statistics, Honeypot Configurator.
- **Styles:** `ghost_trap/style.css`
- **Backend Functions:** `Ghost_Trap_Engine::evaluate()`
- **Данные:** количество сработавших ловушек, заблокированные IP.
- **Действия пользователя:** добавлять дополнительные Decoy Files.

Зависимости

- Вызывает: `VIS_Cerberus::ban_ip()`, `TRINITY_XDR`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`

Модуль: Styx (Zero-Trust Egress Whitelist Guard)

Назначение

Контролирует все исходящие HTTP/HTTPS Connections сервера через `pre_http_request` и препятствует Data Exfiltration, Command-and-Control (C2) Callbacks и DNS Rebinding.

Возможности

- Строгий Whitelist разрешенных Target Domains (WordPress.org Updates, GeDefense Nexus, лицензированные API).
- Блокировка Connections к приватным IP Ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `127.0.0.1`, Cloud Metadata).
- DNS Rebinding Verification до установления Connection.
- Настраиваемый режим: Audit Only (Logging) или Enforce (активная блокировка).

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-styx.php
- includes/dashboard/views/styx/script.js

Backend:
```

```
- includes/modules/styx/class-vis-styx.php

API:
- Filter: 'pre_http_request'

Styles:
- includes/dashboard/views/styx/style.css

Database:
- Options: 'vis_config' (Keys: styx_enabled, styx_mode, styx_whitelist)

Tests:
- scripts/sentinel-threat-benchmark.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `styx`
- **Компоненты:** Egress Traffic Monitor, Whitelist Manager, Block Log.
- **Styles:** `styx/style.css`
- **APIs:** Standard Settings Mutation.
- **Backend Functions:** `VIS_Styx::intercept_request()`, `VIS_Styx::is_whitelisted()`
- **Данные:** исходящие HTTP Requests, заблокированные Exfiltration Attempts.
- **Действия пользователя:** разрешать внешние Target Domains, переключать Mode.

Зависимости

- Требуется Core: `VIS_Security::validate_public_http_url()`
- Отправляет Incidents в: `VIS_Event_Bus`, `TRINITY XDR`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`

Модуль: Titan (Surface Policy Compiler & Browser Confinement)

Назначение

Комплексная система Hardening и Surface Protection. Компилирует динамические контекстные Browser Security Policies (CSP, Fetch Metadata, Permissions Policy, COOP, COEP, HSTS) и защищает чувствительные WordPress Endpoints (XML-RPC, REST Author Enumeration, User Discovery).

Возможности

- Context Resolution девяти Surfaces (`VIS_Titan_Surface_Resolver`): Public, Login, Admin, GeDefense Cockpit, REST, AJAX, Cron, Webhook, Active Preview.
- Детерминированный Compiler для Content Security Policy (CSP Level 3), Strict-Transport-Security (HSTS) и Cross-Origin Policies.
- Строгая 5-stage Lifecycle Model: Candidate Validation -> Report-Only -> Eligibility Check -> Enforce Confirmation -> Rollback.
- Автоматическое получение CSP Violation Reports через внутренний REST API `/wp-json/visiongaia/v1/titan/csp-report`.
- Нативная генерация Server Rules для Nginx и Caddy.
- Встроенное Emergency Recovery через WP-CLI (`VIS_Titan_Recovery`).
- Отключение XML-RPC, Author Enumeration, XML-RPC Honeypot, Emoji Scripts и Version Fingerprints.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-titan.php
- assets/js/vis-titan-command-center.js

Backend:
- includes/modules/titan/class-vis-titan.php
- includes/modules/titan/src/class-titan-surface-resolver.php
- includes/modules/titan/src/class-titan-policy-compiler.php
- includes/modules/titan/src/class-titan-policy-store.php
- includes/modules/titan/src/class-titan-runtime.php
- includes/modules/titan/src/class-titan-assurance.php
- includes/modules/titan/src/class-titan-learning.php
- includes/modules/titan/src/class-titan-violation-collector.php
- includes/modules/titan/src/class-titan-server-rules.php
- includes/modules/titan/src/class-titan-login-gate.php
- includes/modules/titan/src/class-titan-sandbox.php
- includes/modules/titan/src/class-titan-recovery.php

API:
- admin_post_vis_titan_policy_action
```

```
- admin_post_vis_titan_download_nginx
- admin_post_vis_titan_generate_gate_link
- admin_post_vis_titan_preview_link
- REST: POST /wp-json/visiongaia/v1/titan/csp-report

Styles:
- assets/css/vis-titan.css

Database:
- Options: 'vis_config' (Titan Keys), 'vis_titan_policies', 'vis_titan_violations'

Tests:
- scripts/titan-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> titan
- **Компоненты:** Policy Lifecycle Controls, CSP Directive Editor, Violations Viewer, Server Config Download.
- **Styles:** assets/css/vis-titan.css
- **APIs:** admin_post_vis_titan_policy_action, REST Report Ingress.
- **Backend Functions:** VIS_Titan_Runtime::boot(), VIS_Titan_Policy_Compiler::compile()
- **Данные:** активные Browser Headers, Report-Only Hits, заблокированные Domain Origins.
- **Действия пользователя:** активировать/тестировать/откатывать Policies, загружать Server Rules.

Зависимости

- Требуется Core: VIS_Security
- Интегрируется с: Airlock (Certificate Validation)

Используется в

- VIS_Bootstrapper::engage_phase_2()

Модуль: ThroneGuard (Master-Role Privilege Separation)

Назначение

Строго отделяет роль Super Administrator (Master) от обычных WordPress Administrators, отнимает у обычных Admin'ов рискованные Capabilities (Plugin Installation, File Editing) и защищает привилегированные действия криптографическими Superkey Sessions.

Возможности

- Создание Master Role с эксклюзивным контролем над Security Modules.
- Ограничение опасных Capabilities для обычных Admin'ов (install_plugins, edit_plugins, edit_files, update_core).
- Session Superkey Mechanism с Time Limit и Hardware/Fingerprint Binding.
- Неизменяемое Audit Logging всех административных Privilege Activities.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-throneguard.php
- includes/dashboard/views/throneguard/script.js

Backend:
- includes/modules/throneguard/class-vis-throne-guard.php

API:
- admin_post_vis_throneguard_claim
- admin_post_vis_throneguard_save
- admin_post_vis_throneguard_unlock
- wp_ajax_vis_throneguard_clear_logs

Styles:
- includes/dashboard/views/throneguard/style.css

Database:
- Options: 'vis_throneguard_config', 'vis_throneguard_audit_log'

Tests:
- scripts/throneguard-loginpager-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> throneguard (также отдельное Submenu vgt-throneguard)
- **Компоненты:** Master Claim Wizard, Role Matrix, Superkey Unlock Modal, Privilege Audit Log.
- **Styles:** throneguard/style.css

- **APIs:** 3 Admin Post Actions + 1 AJAX Clear Action.
- **Backend Functions:** `VIS_Throne_Guard::handle_claim_master()`, `VIS_Throne_Guard::handle_unlock()`
- **Данные:** Master User ID, Superkey Validity, Role Permissions.
- **Действия пользователя:** заявить Master Role, ограничить Administrator Capabilities, разблокировать Superkey Session.

Зависимости

- Требует Core: `VIS_Vault`, `VIS_Security`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`
- `VIS_Dashboard_Core::register_menu_matrix()`

Модуль: LoginPager (Zero-Trust Login Surface)

Назначение

Предоставляет полностью локальный, укрепленный и визуально современный WordPress Login Interface с Responsive Live Preview в Admin Area и без внешних CDN Dependencies.

Возможности

- Современный Glassmorphic Design стандартной Login Page (`wp-login.php`).
- Protocol-validated подключение Images и Logos (`safe_url()` с HTTP/HTTPS Whitelist).
- Preset Color Schemes: Cyber Цян, Emerald Matrix, Purple Haze, Apex Gold, Crimson Core.
- Полная независимость от внешних Fonts и JavaScript Frameworks.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-loginpager.php
- assets/js/vis-loginpager-admin.js

Backend:
- includes/modules/loginpager/class-vis-loginpager.php

API:
- Mutations via VIS_Dashboard_Settings

Styles:
- includes/dashboard/views/loginpager/style.css

Database:
- Options: 'vis_config' (Keys: loginpager_enabled, loginpager_title, loginpager_subtitle, loginpager_bg_color, loginpager_accent, loginpager_bg_image, loginpager_logo)

Tests:
- scripts/throneguard-loginpager-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `loginpager` (также отдельное Submenu `vgt-loginpager`)
- **Компоненты:** Dual Column Customizer с Live Preview Canvas, Color Picker с Hex Synchronization, Preset Color Swatches.
- **Styles:** `loginpager/style.css`
- **APIs:** Form Mutations через Standard Settings.
- **Backend Functions:** `VIS_LoginPager::render_login_page()`
- **Данные:** Design Tokens, Logo URLs, Branding Texts.
- **Действия пользователя:** настраивать Colors, Background Images и Titles и сразу проверять изменения в Preview.

Зависимости

- Требует Core: `VIS_Security`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`
- `VIS_Dashboard_Core::register_menu_matrix()`

Модуль: Chronos (Autonomous Background Integrity Daemon)

Назначение

Выполняет автономные фоновые проверки всей файловой системы и системной конфигурации через WP-Cron (`vis_hourly_scan_event`) и отправляет предупреждения при обнаружении манипуляций.

Возможности

- Автоматический запуск периодических Integrity и Malware Scans.
- Ресурсосберегающее выполнение с CPU Budgeting.
- Отправка Alert E-Mails при обнаружении подозрительных изменений.
- Самовосстановление прерванных Scan States.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-chronos.php

Backend:
- includes/modules/chronos/class-vis-chronos.php
- includes/modules/chronos/src/class-chronos-engine.php
- includes/modules/chronos/src/class-chronos-scheduler.php
- includes/modules/chronos/src/class-chronos-alerter.php
- includes/modules/chronos/src/class-chronos-config.php

API:
- Hook: 'vis_hourly_scan_event'

Styles:
- includes/dashboard/views/chronos/style.css

Database:
- Options: 'vis_config' (Keys: chronos_enabled, chronos_frequency, chronos_email_recipient, chronos_email_body)

Tests:
- scripts/scanner-resumption-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> chronos
- **Компоненты:** Scheduler Status, Cron Interval Configurator, E-Mail Alert Template Editor.
- **Styles:** chronos/style.css
- **Backend Functions:** `Chronos_Engine::run()`, `Chronos_Alerter::send_alert()`
- **Данные:** Timestamp последнего Scan, запланированный следующий запуск, Alert Recipient.
- **Действия пользователя:** настраивать Scan Frequency, E-Mail Recipient и Template.

Зависимости

- Использует Subsystem: VIS_Scanner_Engine_Omega

Используется в

- `VIS_Bootstrapper::engage_phase_2()`

Модуль: Secure Downloads (Attested Storage & Verifiable Links)

Назначение

Обеспечивает безопасное распространение Media Files и Software Builds через временные криптографически аттестованные GeDefense Download Links со строгим Rate Limiting и защитой от несанкционированного Direct Linking.

Возможности

- Создает неизменяемую изолированную копию файла в защищенном Storage (`VIS_VAULT_DIR`).
- Вычисляет и проверяет SHA-256 Hash при каждом Download Request.
- Отдает файлы через `template_redirect` (-100) по URL `?vis_download={public_id}` без выполнения PHP Files.
- Rate Limiting на Download ID для защиты от DoS.
- Точечный AEGIS WAF Allow Path только для валидированного Download Route.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-downloads.php
```

```
Backend:
- includes/modules/downloads/class-secure-download-manager.php

API:
- Public URL: /?vis_download={public_id}
- admin_post_vis_download_register
- admin_post_vis_download_toggle
- admin_post_vis_download_delete

Styles:
- includes/dashboard/views/downloads/style.css

Database:
- Table: 'wp_vis_secure_downloads'

Tests:
- scripts/download-manager-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> downloads
- **Компоненты:** Registration Form для файлов Media Library, таблица всех активных Downloads с Copy-on-Click.
- **Styles:** downloads/style.css
- **APIs:** 3 Admin Post Actions для Registration, Enable/Pause и Delete.
- **Backend Functions:** DownloadManager::register(), DownloadManager::serve()
- **Данные:** Filename, Integrity Hash, Download Counter, GeDefense Link.
- **Действия пользователя:** защитить файл, копировать Download Link, приостановить/удалить Download.

Зависимости

- Требуется Core: VIS_Schema, VIS_Security
- Пишет в таблицу: wp_vis_secure_downloads

Используется в

- VIS_Bootstrapper::engage_phase_1()
- VIS_Aegis (специализированный Trusted Path)

Модуль: Key Vault (Authenticated AES-256-GCM KMS)

Назначение

Служит защищенным Key Management Service, связанным с сервером/оборудованием, для хранения внешних API Tokens и Secrets с AES-256-GCM Encryption и AAD Salt Binding.

Возможности

- Authenticated Encryption with Associated Data (AAD) против Ciphertext Substitution.
- Key Derivation через HKDF-SHA256 из постоянного Vault Material.
- Строгая изоляция чувствительных Keys в Filesystem с правами 0600.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-vault.php

Backend:
- includes/modules/vault/class-vis-key-vault.php

API:
- Post Actions in View: vis_vault_save, vis_vault_delete

Styles:
- includes/dashboard/views/vault/style.css

Database:
- Options: 'vis_key_vault_registry'

Tests:
- scripts/security-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> vault
- **Компоненты:** Secret Input Form, список sealed Key Assets с Delete Function.
- **Styles:** vault/style.css
- **Backend Functions:** VIS_Key_Vault::seal(), VIS_Key_Vault::unseal()

- **Данные:** Key Identifier, Encryption Mode (AES-256-GCM), Status (SEALED).
- **Действия пользователя:** запечатать новый Secret, необратимо удалить существующий Secret.

Зависимости

- Требуется Core: `VIS_Vault`

Используется в

- `VIS_Bootstrapper::engage_phase_2()`
- `VisionGaiaSE0\VaultBridge`

Модуль: Oracle (12-Vector Security Audit Engine)

Назначение

Выполняет глубокий статический и динамический Security Audit WordPress Installation и выявляет ошибочные конфигурации в 12 ключевых областях.

Возможности

- Проверка File Permissions, SSL Configuration, Debug Logs, User Privileges, DB Prefixes и устаревшего Software.
- Генерация структурированных Audit Findings с рекомендациями.
- Встроенный Interface для AI-assisted Vulnerability Assessment.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-oracle.php

Backend:
- includes/modules/oracle/class-vis-oracle.php

API:
- wp_ajax_vis_oracle_ping

Styles:
- includes/dashboard/views/oracle/style.css

Database:
- Table: 'wp_vis_oracle_patterns'

Tests:
- scripts/dashboard-consolidation-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> `oracle`
- **Компоненты:** 12-vector Audit Matrix, Vector Status Cards (Pass/Fail), Recommendations.
- **Styles:** `oracle/style.css`
- **APIs:** `wp_ajax_vis_oracle_ping`
- **Backend Functions:** `VIS_Oracle::get_audit_findings()`
- **Данные:** System Integrity Score, Vector Findings.
- **Действия пользователя:** вручную запускать Scan, анализировать Vulnerabilities.

Зависимости

- Требуется Core: `VIS_Security_Health`

Используется в

- `VIS_Bootstrapper::engage_phase_1()`

Модуль: Filesystem Guard (Permission & CHMOD Invariant Auditor)

Назначение

Контролирует Filesystem Permissions критических компонентов WordPress и подает Alert, если чувствительные файлы получают избыточные права записи.

Возможности

- Проверка CHMOD Rights для `wp-config.php` (ожидается 0400/0440/0600), `.htaccess` и Directories.
- Обнаружение незащищенных Directories без `index.php` или `.htaccess`.

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-filesystem.php

Backend:
- includes/modules/filesystem/class-vis-filesystem-guard.php

Styles:
- includes/dashboard/views/filesystem/style.css
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> filesystem
- **Компоненты:** File Permission Table со Status Badges (Safe / Warning).
- **Styles:** filesystem/style.css
- **Backend Functions:** VIS_FileSystem_Guard::audit()
- **Данные:** Path, текущий CHMOD, рекомендуемый CHMOD, Security Status.

Зависимости

- Требуется Core: VIS_Security::jailed_path()

Используется в

- VIS_Bootstrapper::engage_phase_2()

Модуль: Kernel Sentinel (Runtime Invariant Monitor)

Назначение

Контролирует Low-Level PHP Runtime Environment и Kernel Invariants (Memory Limit, Max Execution Time, OPcache Status) и обеспечивает раннее предупреждение о ресурсных ограничениях.

Возможности

- Monitoring Memory Usage и Peak Memory.
- Проверка PHP Extensions (Sodium, OpenSSL, BCMath, ZipArchive).

Связанные файлы

```
Frontend / View:
- includes/dashboard/views/view-kernel.php
- includes/dashboard/views/kernel/script.js

Backend:
- includes/modules/kernel/class-vis-kernel-sentinel.php

Styles:
- includes/dashboard/views/kernel/style.css
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> kernel
- **Компоненты:** Kernel Status Display, Memory Usage Graph, System Parameters.
- **Styles:** kernel/style.css
- **Script:** kernel/script.js
- **Backend Functions:** VIS_Kernel_Sentinel::get_system_report()

Зависимости

- Требуется Core: VIS_Security_Health

Используется в

- VIS_Bootstrapper::engage_phase_2()

Подсистема: TRINITY XDR Fabric

Назначение

Extended Detection & Response (XDR) Fabric объединяет Event Streams всех Sensors (Cerberus, Aegis, Zeus, Prometheus, Nemesis, Morpheus), коррелирует отдельные атаки в связанные Incidents, формирует понятные Attack Stories и оркестрирует обратимые Countermeasures с audit-safe Merkle Evidence.

Возможности

- Типизированная Security Event Model (XdrEvent) с Severity, Attribution Score, Actor Hash и Privacy Masking.
- Причинная Incident Correlation (IncidentEngine) по IP, Session и временным окнам.
- Rule-based Policy Evaluation (PolicyEngine) для автоматических Responses.
- Reversible Response Orchestration (ResponseEngine): IP Bans, Session Termination, Quarantine с полным Rollback Support.
- Merkle Evidence Chain (EvidenceStore): криптографически связанная Digest History для каждого Incident в wp_vis_xdr_evidence.

Связанные файлы

```
Backend:
- includes/xdr/class-xdr-event-fabric.php
- includes/xdr/class-xdr-event.php
- includes/xdr/class-xdr-event-repository.php
- includes/xdr/class-xdr-incident-engine.php
- includes/xdr/class-xdr-policy-engine.php
- includes/xdr/class-xdr-response-engine.php
- includes/xdr/class-xdr-evidence-store.php
- includes/xdr/class-xdr-redactor.php
- includes/xdr/class-xdr-request-context.php

Dashboard / Frontend:
- includes/dashboard/views/view-trinity.php
- includes/dashboard/views/view-xdr.php
- includes/dashboard/views/view-thread.php
- includes/dashboard/views/trinity/style.css
- includes/dashboard/views/thread/style.css
- includes/dashboard/views/xdr/style.css
- assets/css/vis-xdr.css
- includes/dashboard/views/thread/script.js

Database:
- wp_vis_xdr_events
- wp_vis_xdr_incidents
- wp_vis_xdr_incident_events
- wp_vis_xdr_responses
- wp_vis_xdr_evidence

Tests:
- scripts/xdr-regression.php
- scripts/xdr-adversarial-regression.php
- scripts/xdr-release-blockers-regression.php
```

Интеграция с Dashboard

- **Страницы:** Dashboard -> trinity, thread, xdr
- **Компоненты:** Live Situation View, Incident Cards с Attack Story, Response History с Rollback Button, криптографические Evidence Certificates.
- **Данные:** структурированные Events, Threat Clusters, автоматические Countermeasures.

Подсистема: Scanner & Malware Engine

Назначение

Предоставляет непрерывные, пагинированные и ограниченные по ресурсам File Integrity/Malware Scans для всей WordPress Filesystem с SHA-256 Merkle Tree Baselines и защищенным Quarantine Vault.

Возможности

- Возобновляемые Scans (VIS_Scanner_Engine_Omega) через NDJSON State Files в защищенном Vault.
- File Detection через AST Lexical Flow, Polyglot Detection и SVG/XML Sanitization.
- Quarantine зараженных файлов в VIS_VAULT_DIR/quarantine/ с AES-256-GCM Encryption.

Связанные файлы

```
Backend:
- includes/scanner/class-vis-scanner-engine.php
- includes/scanner/class-vis-malware-engine.php
- includes/scanner/detectors/class-vis-php-lexical-detector.php
- includes/scanner/detectors/class-vis-polyglot-detector.php
- includes/scanner/detectors/class-vis-svg-xml-detector.php
- includes/scanner/detectors/class-vis-archive-detector.php
- includes/scanner/detectors/class-vis-path-context-detector.php
- includes/scanner/storage/class-vis-quarantine-store.php
- includes/scanner/contracts/interface-vis-file-detector.php
- includes/scanner/value/class-vis-scan-budget.php
- includes/scanner/value/class-vis-scan-context.php
- includes/scanner/value/class-vis-scan-finding.php
- includes/scanner/value/class-vis-scan-verdict.php
```



```

Frontend:
- assets/js/vis-scanner-client.js
- assets/css/vis-scanner-modal.css
- includes/dashboard/views/view-integrity.php
- includes/dashboard/views/integrity/script.js
- includes/dashboard/views/integrity/style.css

API:
- wp_ajax_vis_run_scan
- wp_ajax_vgt_integrity_uplink
- wp_ajax_vis_inspect_file
- wp_ajax_vis_approve_changes

Tests:
- scripts/scanner-resumption-regression.php
- scripts/integrity-baseline-regression.php
- scripts/malware-scanner-regression.php

```

Динамический Add-On: Vision Legal Pro (VLP)

Назначение

Расширение для Data Protection и Compliance: локальный Asset Mirroring (Google Fonts, JavaScript) через Shadow-Net Proxy, Cookie Consent Management и AI-assisted Multilingual Translation (Lingua AI).

Возможности

- Shadow-Net: перехватывает External Resource Requests и зеркалирует Assets локально без утечки данных на US Servers.
- Privacy Shield: управляет Cookie Consent и блокирует Trackers до получения Consent.
- Lingua AI: автоматически переводит Website Content через Groq LLM.

Связанные файлы

```

Backend:
- includes/VLP/vision-legal-pro.php
- includes/VLP/includes/core/class-vlp-asset-library.php
- includes/VLP/includes/core/class-vlp-service-definitions.php
- includes/VLP/includes/admin/class-vlp-admin-dashboard.php
- includes/VLP/includes/modules/shadow-net/
- includes/VLP/includes/modules/privacy-shield/
- includes/VLP/includes/modules/lingua/
- includes/VLP/includes/modules/dattrack/

Dashboard:
- includes/dashboard/views/view-vlp.php
- includes/dashboard/views/vlp/style.css
- includes/dashboard/views/vlp/script.js

API:
- admin_post_vgt_sync
- admin_post_vgt_export_csv
- admin_post_vgt_export_pdf
- wp_ajax_vlp_save_banner
- wp_ajax_vlp_download_asset_secure

```

Динамический Add-On: VisionGaia SEO Architect

Назначение

Полноценная SEO Suite: автоматическая Metadata Generation, XML Sitemaps, 301 Redirect Management, локальные Schema.org Geo Injections и Relevance Analysis.

Возможности

- Автоматическая Metatag Generation через AI.
- Высокопроизводительный Sitemap Generator без Third-Party Plugins.
- Встроенный 301/302 Redirect Manager с Loop Detection.

Связанные файлы

```

Backend:
- includes/VisionGaiaSEO/visiongaia-seo-architect.php
- includes/VisionGaiaSEO/class-vg-seo-bootstrapper.php
- includes/VisionGaiaSEO/class-vg-seo-ajax-router.php
- includes/VisionGaiaSEO/class-vg-seo-settings.php
- includes/VisionGaiaSEO/includes/class-vg-api-service.php

```

```
- includes/VisionGaiaSEO/includes/class-vg-automation-engine.php
- includes/VisionGaiaSEO/includes/class-vg-geo-injector.php
- includes/VisionGaiaSEO/includes/class-vg-seo-relevance.php
- includes/VisionGaiaSEO/includes/class-vg-vault-bridge.php
- includes/VisionGaiaSEO/includes/class-vgt-data-manager.php
- includes/VisionGaiaSEO/includes/class-vgt-frontend-renderer.php
- includes/VisionGaiaSEO/includes/class-vgt-meta.php
- includes/VisionGaiaSEO/includes/class-vgt-redirect.php
- includes/VisionGaiaSEO/includes/class-vgt-sitemap.php
```

API:

```
- wp_ajax_vg_seo_generate
- wp_ajax_vg_seo_reset_stats
- wp_ajax_vg_seo_get_all_pending
- wp_ajax_vg_seo_get_all_completed
- wp_ajax_vgt_add_redirect
- wp_ajax_vgt_delete_redirect
- admin_post_vgt_vault_save
- admin_post_vgt_vault_delete
```

Tests:

```
- scripts/seo-relevance-regression.php
```

Динамический Add-On: VisionGaia Builder

Назначение

Встроенный Visual Editor и Migrator: конвертирует Elementor Pages в легкий нативный GeDefense Code и предоставляет визуальное Live Editing с AI Copilot.

Возможности

- Bulk Migration Elementor Posts в стандартный HTML/CSS.
- Изолированный Live Preview Broker для безопасного отображения.
- AI Copilot для генерации Responsive Layouts.

Связанные файлы

Backend:

```
- includes/builder/builder.php
- includes/builder/inc/class-vgt-admin.php
- includes/builder/inc/class-vgt-ajax.php
- includes/builder/inc/class-vgt-content-gate.php
- includes/builder/inc/class-vgt-frontend.php
- includes/builder/inc/class-vgt-migration.php
- includes/builder/inc/class-vgt-preview-broker.php
- includes/builder/inc/class-vgt-telemetry.php
```

Views:

```
- includes/builder/views/dashboard.php
- includes/builder/views/editor-ui.php
- includes/builder/views/bulk-migration.php
```

API:

```
- wp_ajax_vgt_render_preview
- wp_ajax_vgt_copilot_generate
- wp_ajax_vgt_migrate_elementor
- wp_ajax_vgt_get_elementor_pages
- wp_ajax_vgt_process_bulk_migration
```

Tests:

```
- scripts/builder-live-preview-regression.php
```

4. ПОДРОБНАЯ ДОКУМЕНТАЦИЯ DASHBOARD

В GeDefense Cockpit каждая страница маршрутизируется через центральную страницу меню `admin.php?page=vgt-suite&tab={tab}`. Rendering выполняет `VIS_Dashboard_View::render()`, а подключением Assets управляет `VIS_Dashboard_Assets::enqueue()`.

Dashboard -> Overview (Control Center)

- **Route:** `admin.php?page=vgt-suite&tab=overview`
- **Frontend File:** `includes/dashboard/views/view-overview.php`

- **CSS / Styling:** includes/dashboard/views/overview/style.css, assets/css/vis-dashboard.css, assets/css/vis-dashboard-modern.css
- **Используемые компоненты:** Vital HUD со Score Gauges, System Quick Check Matrix, Live Ban List Extract, Scan Trigger Button.
- **Ответственный модуль:** Core Cockpit / Health Engine (VIS_Security_Health, VIS_Security_Center)
- **API Calls:** wp_ajax_vis_run_scan, wp_ajax_vis_dashboard_unban_ip
- **Backend Handler:** VIS_Dashboard_Ajax::handle_scan_bridge(), VIS_Dashboard_Ajax::handle_unban_ip()
- **Services:** VIS_Security_Health::evaluate(), VIS_Scanner_Engine_Omega
- **Database Access:** читает wp_vis_apex_bans, wp_vis_omega_logs, wp_vis_xdr_incidents
- **Authentication/Permissions:** current_user_can('manage_options'), Nonce: vis_nonce
- **Поток данных:**

```

User открывает Overview
↓
view-overview.php загружает Vital Status и Log Summary
↓
GET из wp_vis_apex_bans, wp_vis_omega_logs, wp_vis_xdr_incidents
↓
HTML Rendering (Server-Side)
↓
[Optional] User нажимает "Проверить целостность системы"
↓
overview/script.js отправляет AJAX 'vis_run_scan'
↓
VIS_Dashboard_Ajax::handle_scan_bridge()
↓
VIS_Scanner_Engine_Omega::run_batch()
↓
JSON Response с Progress и Findings
↓
UI обновляется в Scan Modal

```

Dashboard -> Thread (Threat Matrix)

- **Route:** admin.php?page=vgt-suite&tab=thread
- **Frontend File:** includes/dashboard/views/view-thread.php
- **CSS / Styling:** includes/dashboard/views/thread/style.css
- **Используемые компоненты:** XDR Event Feed, Filter Bar (Severity, Module, IP), Detail Inspector.
- **Ответственный модуль:** TRINITY XDR / Prometheus
- **API Calls:** Client-side DOM Filtering через thread/script.js
- **Backend Handler:** Server-Side Rendering через PHP
- **Database Access:** читает wp_vis_omega_logs, wp_vis_xdr_events
- **Authentication/Permissions:** manage_options

Dashboard -> Oracle (Oracle Scanner)

- **Route:** admin.php?page=vgt-suite&tab=oracle
- **Frontend File:** includes/dashboard/views/view-oracle.php
- **CSS / Styling:** includes/dashboard/views/oracle/style.css
- **Используемые компоненты:** 12-vector Audit Matrix, Anomaly Radar, Remediation Guide.
- **Ответственный модуль:** Oracle (VIS_Oracle)
- **API Calls:** wp_ajax_vis_oracle_ping
- **Backend Handler:** VIS_Dashboard_Ajax::handle_oracle_ping()
- **Services:** VIS_Oracle::get_audit_findings()
- **Database Access:** читает/пишет wp_vis_oracle_patterns
- **Authentication/Permissions:** manage_options, Nonce: vis_oracle_ping

Dashboard -> Integrity (Integrity Monitor)

- **Route:** admin.php?page=vgt-suite&tab=integrity
- **Frontend File:** includes/dashboard/views/view-integrity.php
- **CSS / Styling:** includes/dashboard/views/integrity/style.css
- **Используемые компоненты:** Merkle Tree Hash Status, File Explorer Modal с Syntax Highlighting, Approval Trigger для Modifications.

- **Ответственный модуль:** Core Module Integrity (VIS_Module_Integrity) & Scanner Engine
- **API Calls:** wp_ajax_vis_inspect_file, wp_ajax_vis_approve_changes, wp_ajax_vis_run_scan
- **Backend Handler:** VIS_Dashboard_Ajax::handle_inspect_file(), VIS_Dashboard_Ajax::handle_approve()
- **Services:** VIS_Module_Integrity::verify_all(), VIS_Quarantine_Store
- **Database Access:** Options: vis_file_hashes, файл integrity/module-manifest.json
- **Authentication/Permissions:** manage_options, Nonce: vis_nonce

Dashboard -> Security Center

- **Route:** admin.php?page=vgt-suite&tab=security_center (&security_section=assurance|system|logs)
- **Frontend File:** includes/dashboard/views/view-security_center.php (встраивает view-systatus.php и view-logs.php)
- **CSS / Styling:** assets/css/vis-security-center.css, systatus/style.css, logs/style.css
- **Используемые компоненты:** Sub Navigation, Self-Test Trigger, System Diagnostics Metrics, Logbook с Search/Filter Bar.
- **Ответственный модуль:** Security Center (VIS_Security_Center), Security Health (VIS_Security_Health), Event Bus
- **API Calls:** wp_ajax_vis_security_center_test, wp_ajax_vis_dashboard_unban_ip
- **Backend Handler:** VIS_Dashboard_Ajax::handle_security_center_test(), VIS_Dashboard_Ajax::handle_unban_ip()
- **Services:** VIS_Security_Center::run_diagnostics(), VIS_Security_Health::evaluate()
- **Database Access:** читает wp_vis_omega_logs, wp_vis_apex_bans, wp_vis_xdr_events
- **Authentication/Permissions:** manage_options, Nonce: vis_nonce

Dashboard -> Trinity (TRINITY XDR)

- **Route:** admin.php?page=vgt-suite&tab=trinity
- **Frontend File:** includes/dashboard/views/view-trinity.php
- **CSS / Styling:** includes/dashboard/views/trinity/style.css, assets/css/vis-xdr.css
- **Используемые компоненты:** 4-pillar Interlock HUD (Cerberus, Aegis, Prometheus, Nemesis), Vector Distribution Pie, Quick Configuration Switches.
- **Ответственный модуль:** TRINITY XDR / Trinity Grid (VIS_Trinity_Grid)
- **API Calls:** Form Mutations через Settings
- **Backend Handler:** VIS_Dashboard_Settings::process_mutations()
- **Database Access:** читает wp_vis_xdr_events, wp_vis_xdr_incidents, wp_vis_apex_bans
- **Authentication/Permissions:** manage_options, Nonce: vis_save_config

Dashboard -> Zeus (Zeus Defender)

- **Route:** admin.php?page=vgt-suite&tab=zeus
- **Frontend File:** includes/dashboard/views/view-zeus.php
- **CSS / Styling:** includes/dashboard/views/zeus/style.css
- **Используемые компоненты:** WAF Rule Matrix, Latency Benchmark Terminal, Blackbox Drain Monitor, Admission Token Generator, Preset Selector.
- **Ответственный модуль:** Zeus (VIS_Zeus)
- **API Calls:** 9 AJAX Endpoints (vis_save_zeus_config, vis_zeus_run_benchmark, etc.)
- **Backend Handler:** VIS_Dashboard_Ajax::handle_zeus_*
- **Services:** Zeus_Compiler, Zeus_Benchmark, Zeus_Admission
- **Database Access:** Options: vis_zeus_config, vis_zeus_contracts
- **Authentication/Permissions:** manage_options, Nonce: vis_nonce

Dashboard -> Aegis (AEGIS Firewall)

- **Route:** admin.php?page=vgt-suite&tab=aegis
- **Frontend File:** includes/dashboard/views/view-aegis.php
- **CSS / Styling:** includes/dashboard/views/aegis/style.css

- **Используемые компоненты:** Mode Switch (Monitor/Enforce), Whitelist Textareas для IP и User Agents, Diagnostics Terminal.
 - **Ответственный модуль:** Aegis (VIS_Aegis)
 - **API Calls:** wp_ajax_vis_oracle_ping (через vis-oracle-diagnostics.js)
 - **Backend Handler:** VIS_Dashboard_Settings::process_mutations(), VIS_Dashboard_Ajax::handle_oracle_ping()
 - **Database Access:** Options: vis_config (Keys: aegis_enabled, aegis_mode, aegis_whitelist_ips, aegis_whitelist_uas)
 - **Authentication/Permissions:** manage_options, Nonce: vis_save_config
-

Dashboard -> Prometheus (Prometheus Engine)

- **Route:** admin.php?page=vgt-suite&tab=prometheus
 - **Frontend File:** includes/dashboard/views/view-prometheus.php
 - **CSS / Styling:** includes/dashboard/views/prometheus/style.css
 - **Используемые компоненты:** Subnet Risk Heatmap, Threshold Slider, IP Whitelist.
 - **Ответственный модуль:** Prometheus (VIS_Prometheus)
 - **API Calls:** Client Filter через prometheus/script.js, Form Save через Settings
 - **Database Access:** Options: vis_config (Prometheus Keys)
 - **Authentication/Permissions:** manage_options, Nonce: vis_save_config
-

Dashboard -> Cerberus (Cerberus IP Ban)

- **Route:** admin.php?page=vgt-suite&tab=cerberus
 - **Frontend File:** includes/dashboard/views/view-cerberus.php
 - **CSS / Styling:** includes/dashboard/views/cerberus/style.css
 - **Используемые компоненты:** Live Ban List с Pagination, Unban Action, Manual Ban Form, Server Rule Download.
 - **Ответственный модуль:** Cerberus (VIS_Cerberus)
 - **API Calls:** wp_ajax_vis_dashboard_unban_ip
 - **Backend Handler:** VIS_Dashboard_Ajax::handle_unban_ip()
 - **Services:** VIS_Cerberus::instance()->unban_ip()
 - **Database Access:** читает/удаляет wp_vis_apex_bans
 - **Authentication/Permissions:** manage_options, Nonce: vis_nonce
-

Dashboard -> Airlock (Airlock Gateway)

- **Route:** admin.php?page=vgt-suite&tab=airlock
 - **Frontend File:** includes/dashboard/views/view-airlock.php
 - **CSS / Styling:** includes/dashboard/views/airlock/style.css
 - **Используемые компоненты:** File Inspection Status Cards, SVG Sanitization Toggle, Quarantine Options.
 - **Ответственный модуль:** Airlock (VIS_Airlock)
 - **API Calls:** Form Mutations через Settings
 - **Database Access:** Options: vis_config (Airlock Keys)
 - **Authentication/Permissions:** manage_options, Nonce: vis_save_config
-

Dashboard -> Nemesis (Nemesis Deception)

- **Route:** admin.php?page=vgt-suite&tab=nemesis
 - **Frontend File:** includes/dashboard/views/view-nemesis.php
 - **CSS / Styling:** includes/dashboard/views/nemesis/style.css
 - **Используемые компоненты:** Tarpit Latency Configurator, Fake Response Selector, Honeytoken Audit.
 - **Ответственный модуль:** Nemesis (VIS_Nemesis)
 - **API Calls:** Settings Mutation
 - **Database Access:** Options: vis_config (Nemesis Keys)
 - **Authentication/Permissions:** manage_options, Nonce: vis_save_config
-

Dashboard -> Ghost Trap (Ghost Honeypot)

- **Route:** `admin.php?page=vgt-suite&tab=ghost_trap`
- **Frontend File:** `includes/dashboard/views/view-ghost_trap.php`
- **CSS / Styling:** `includes/dashboard/views/ghost_trap/style.css`
- **Используемые компоненты:** список активных Decoy Files (`.env`, `wp-config.php.bak`), Hit Statistics.
- **Ответственный модуль:** Ghost Trap (`VIS_Ghost_Trap`)
- **API Calls:** Settings Mutation
- **Database Access:** Options: `vis_config` (Keys: `ghost_trap_enabled`, `ghost_trap_exts`)
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_save_config`

Dashboard -> Hades (Hades Stealth)

- **Route:** `admin.php?page=vgt-suite&tab=hades`
- **Frontend File:** `includes/dashboard/views/view-hades.php`
- **CSS / Styling:** `includes/dashboard/views/hades/style.css`
- **Используемые компоненты:** генератор секретного Login URL, блок правил Rewrite для Nginx/Apache.
- **Ответственный модуль:** Hades (`VIS_Hades`)
- **API Calls:** Settings Mutation
- **Database Access:** Options: `vis_config` (Hades Keys)
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_save_config`

Dashboard -> Morpheus (Morpheus Sandbox)

- **Route:** `admin.php?page=vgt-suite&tab=morpheus`
- **Frontend File:** `includes/dashboard/views/view-morpheus.php`
- **CSS / Styling:** `includes/dashboard/views/morpheus/style.css`
- **Используемые компоненты:** Live Callstack Inspector, таблица Hook Audit с действиями разрешения/блокировки, AI Stack Analysis.
- **Ответственный модуль:** Morpheus (`Vis_Morpheus`)
- **API Calls:** 5 AJAX endpoints (`vgt_morpheus_trigger_ai`, etc.)
- **Backend Handler:** `Morpheus_Dashboard::ajax_*`
- **Database Access:** Options: `vgt_morpheus_audit_matrix`, `vis_config`
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_nonce / vgt_morpheus_isolation`

Dashboard -> Titan (Titan Hardening)

- **Route:** `admin.php?page=vgt-suite&tab=titan`
- **Frontend File:** `includes/dashboard/views/view-titan.php`
- **CSS / Styling:** `assets/css/vis-titan.css`
- **Используемые компоненты:** панель управления 5-этапным Policy Lifecycle, редактор CSP directives, Violations Viewer, загрузка server config.
- **Ответственный модуль:** Titan (`VIS_Titan`)
- **API Calls:** `admin_post_vis_titan_policy_action`, `admin_post_vis_titan_download_nginx`, `admin_post_vis_titan_generate_gate_link`
- **Backend Handler:** `VIS_Titan::handle_policy_action()`, `VIS_Titan_Server_Rules::handleDownload()`
- **Database Access:** Options: `vis_config` (Titan Keys), `vis_titan_policies`, `vis_titan_violations`
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_titan_policy_action`

Dashboard -> Kernel (Kernel Uplink)

- **Route:** `admin.php?page=vgt-suite&tab=kernel`
- **Frontend File:** `includes/dashboard/views/view-kernel.php`
- **CSS / Styling:** `includes/dashboard/views/kernel/style.css`
- **Используемые компоненты:** низкоуровневые системные параметры, статус OPcache, индикатор Memory Peak.
- **Ответственный модуль:** Kernel Sentinel (`VIS_Kernel_Sentinel`)

- **Database Access:** прямые операции записи в БД отсутствуют.
- **Authentication/Permissions:** `manage_options`

Dashboard -> Styx (Styx Controller)

- **Route:** `admin.php?page=vgt-suite&tab=styx`
- **Frontend File:** `includes/dashboard/views/view-styx.php`
- **CSS / Styling:** `includes/dashboard/views/styx/style.css`
- **Используемые компоненты:** журнал исходящего трафика, редактор доменного Whitelist.
- **Ответственный модуль:** Styx (`VIS_Styx`)
- **API Calls:** Settings Mutation
- **Database Access:** Options: `vis_config` (Styx Keys)
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_save_config`

Dashboard -> Chronos (Chronos Autopilot)

- **Route:** `admin.php?page=vgt-suite&tab=chronos`
- **Frontend File:** `includes/dashboard/views/view-chronos.php`
- **CSS / Styling:** `includes/dashboard/views/chronos/style.css`
- **Используемые компоненты:** статус WP-Cron Scheduler, выбор частоты (Hourly/Daily), редактор e-mail alerts.
- **Ответственный модуль:** Chronos (`VIS_Chronos`)
- **API Calls:** Settings Mutation
- **Database Access:** Options: `vis_config` (Chronos Keys)
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_save_config`

Dashboard -> VLP (Data Protection & Shadow-Net)

- **Route:** `admin.php?page=vgt-suite&tab=vlp`
- **Frontend File:** `includes/dashboard/views/view-vlp.php`
- **CSS / Styling:** `includes/dashboard/views/vlp/style.css`
- **Используемые компоненты:** статус Shadow-Net Proxy, локальный каталог assets, настройки GDPR Shield.
- **Ответственный модуль:** Vision Legal Pro Bridge (`VIS_Bridge_Vision_Legal_Pro`) / VLP
- **API Calls:** `admin_post_vgt_sync`, `wp_ajax_vlp_download_asset_secure`
- **Database Access:** Options: `vis_vlp_config`
- **Authentication/Permissions:** `manage_options`

Dashboard -> Filesystem (File Security)

- **Route:** `admin.php?page=vgt-suite&tab=filesystem`
- **Frontend File:** `includes/dashboard/views/view-filesystem.php`
- **CSS / Styling:** `includes/dashboard/views/filesystem/style.css`
- **Используемые компоненты:** таблица CHMOD permissions для ключевых файлов с цветовой индикацией состояния.
- **Ответственный модуль:** Filesystem Guard (`VIS_Filesystem_Guard`)
- **Backend Handler:** Server-Side Rendering
- **Database Access:** отсутствует - проверяется непосредственно файловая система.
- **Authentication/Permissions:** `manage_options`

Dashboard -> Vault (Key Vault)

- **Route:** `admin.php?page=vgt-suite&tab=vault`
- **Frontend File:** `includes/dashboard/views/view-vault.php`
- **CSS / Styling:** `includes/dashboard/views/vault/style.css`
- **Используемые компоненты:** ввод секрета (Key-ID & Password), registry table зашифрованных ключей с функцией удаления.

- **Ответственный модуль:** Key Vault (VIS_Key_Vault)
- **API Calls:** Post Actions: vis_vault_save, vis_vault_delete
- **Backend Handler:** VIS_Key_Vault::handle_post()
- **Database Access:** Options: vis_key_vault_registry, зашифрованные blobs.
- **Authentication/Permissions:** manage_options, Nonces: vis_vault_save_action, vis_vault_delete_action

Dashboard -> ThroneGuard

- **Route:** admin.php?page=vgt-throneguard или admin.php?page=vgt-suite&tab=throneguard
- **Frontend File:** includes/dashboard/views/view-throneguard.php
- **CSS / Styling:** includes/dashboard/views/throneguard/style.css
- **Используемые компоненты:** Master Admin Claiming Card, таблица role permissions, Superkey Session Unlocker, Audit Log.
- **Ответственный модуль:** ThroneGuard (VIS_Throne_Guard)
- **API Calls:** admin_post_vis_throneguard_claim, admin_post_vis_throneguard_save, admin_post_vis_throneguard_unlock, wp_ajax_vis_throneguard_clear_logs
- **Backend Handler:** VIS_Throne_Guard::handle_*
- **Database Access:** Options: vis_throneguard_config, vis_throneguard_audit_log
- **Authentication/Permissions:** manage_options, Nonces: vis_throneguard_nonce, vis_throneguard_claim_nonce, vis_throneguard_unlock_nonce

Dashboard -> LoginPager

- **Route:** admin.php?page=vgt-loginpager или admin.php?page=vgt-suite&tab=loginpager
- **Frontend File:** includes/dashboard/views/view-loginpager.php
- **CSS / Styling:** includes/dashboard/views/loginpager/style.css
- **Используемые компоненты:** двухколоночная Studio UI с интерактивным Live Preview страницы входа, Preset Swatches, Color Picker с Hex Input, Image URL Binding.
- **Ответственный модуль:** LoginPager (VIS_LoginPager)
- **API Calls:** Settings Mutation через форму.
- **Backend Handler:** VIS_Dashboard_Settings::process_mutations()
- **Database Access:** Options: vis_config (LoginPager Keys)
- **Authentication/Permissions:** manage_options, Nonce: vis_save_config

Dashboard -> Downloads (Secure Downloads)

- **Route:** admin.php?page=vgt-suite&tab=downloads
- **Frontend File:** includes/dashboard/views/view-downloads.php
- **CSS / Styling:** includes/dashboard/views/downloads/style.css
- **Используемые компоненты:** Media Library selector для создания ссылок, таблица активных downloads с copy-on-click URL и счетчиком.
- **Ответственный модуль:** Secure Downloads (DownloadManager)
- **API Calls:** admin_post_vis_download_register, admin_post_vis_download_toggle, admin_post_vis_download_delete
- **Backend Handler:** DownloadManager::registerAction(), toggleAction(), deleteAction()
- **Database Access:** читает/записывает wp_vis_secure_downloads.
- **Authentication/Permissions:** manage_options, Nonces: vis_download_register, vis_download_toggle, vis_download_delete

Dashboard -> Modules (Add-On Management)

- **Route:** admin.php?page=vgt-suite&tab=modules
- **Frontend File:** includes/dashboard/views/view-modules.php
- **CSS / Styling:** includes/dashboard/views/modules/style.css
- **Используемые компоненты:** Drag-and-Drop ZIP uploader, grid установленных Add-Ons (VLP, Builder, SEO) с кнопками удаления.
- **Ответственный модуль:** Module Registry (VIS_Module_Registry)

- **API Calls:** `wp_ajax_vis_upload_addon`, `wp_ajax_vis_uninstall_addon`
- **Backend Handler:** `VIS_Dashboard_Ajax::handle_upload_addon()`, `VIS_Dashboard_Ajax::handle_uninstall_addon()`
- **Services:** `VIS_Module_Registry::all()`, проверка безопасности `ZipArchive` (защита от Zip Bomb и Path Traversal).
- **Database Access:** Options: `vis_config`
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_nonce`

Dashboard -> Setup Wizard

- **Route:** `admin.php?page=vgt-suite&tab=setup_wizard`
- **Frontend File:** `includes/dashboard/views/view-setup_wizard.php`
- **CSS / Styling:** `includes/dashboard/views/setup_wizard/style.css`
- **Используемые компоненты:** 7-step assistant (Admin IP Whitelist, protection level, WAF, RASP, Deception, Hardening, completion).
- **Ответственный модуль:** Dashboard Setup Flow
- **API Calls:** Settings Mutation с автоматическим завершением (`vgt_setup_wizard_completed = 1`).
- **Backend Handler:** `VIS_Dashboard_Settings::process_mutations()`
- **Services:** `VIS_Security::client_ip()`
- **Database Access:** Options: `vis_config`, `vgt_setup_wizard_completed`
- **Authentication/Permissions:** `manage_options`, Nonce: `vis_save_config`

Dashboard -> XDR (Situation View & Incident Deep Dive)

- **Route:** `admin.php?page=vgt-suite&tab=xdr`
- **Frontend File:** `includes/dashboard/views/view-xdr.php`
- **CSS / Styling:** `includes/dashboard/views/xdr/style.css`, `assets/css/vis-xdr.css`
- **Используемые компоненты:** детальная Situation View, Attack Stories, Causal Event Graph, trigger Response Rollback, Evidence Root Inspector.
- **Ответственный модуль:** TRINITY XDR (`VisionGaia\GeDefense\Xdr\*`)
- **Database Access:** `wp_vis_xdr_incidents`, `wp_vis_xdr_events`, `wp_vis_xdr_responses`, `wp_vis_xdr_evidence`
- **Authentication/Permissions:** `manage_options`

Dashboard -> Gorgon (Neural Sync Matrix)

- **Route:** `admin.php?page=vgt-suite&tab=gorgon`
- **Frontend File:** `includes/dashboard/views/view-gorgon.php`
- **CSS / Styling:** `includes/dashboard/views/gorgon/style.css`
- **Используемые компоненты:** статус Mesh nodes, журнал синхронизации, Peer Configurator.
- **Ответственный модуль:** Gorgon (`VisionGaia\GeDefense\Modules\Gorgon\Gorgon`)
- **API Calls:** 6 AJAX routes (`vgt_gorgon_*`)
- **Backend Handler:** `Gorgon_Ajax::handle_*`
- **Database Access:** Options: `vis_config` (Gorgon Keys)
- **Authentication/Permissions:** `manage_options`, Nonce: `vgt_gorgon_nonce`

5. CSS / UI АРХИТЕКТУРА

Вся UI design system GeDefense основана на **Glassmorphism Dark-Mode Matrix** и использует нативные CSS variables (Design Tokens). **Внешние CSS frameworks и webfonts не используются.**

5.1 Глобальные Design Tokens и переменные

Определены в `assets/css/vis-dashboard.css` и дополнительно уточняются в `assets/css/vis-dashboard-modern.css`:

```
:root {
  --vgt-bg-base: #070a13;
  --vgt-bg-panel: rgba(13, 18, 36, 0.7);
```

```

--vgt-bg-panel-elevated: rgba(20, 27, 54, 0.85);
--vgt-border: rgba(255, 255, 255, 0.08);
--vgt-border-hover: rgba(255, 255, 255, 0.2);
--vgt-text-main: #f8fafc;
--vgt-text-dim: #94a3b8;
--vgt-text-muted: #64748b;
--vgt-neon-purple: #a855f7;
--vgt-neon-green: #10b981;
--vgt-neon-red: #ef4444;
--vgt-neon-blue: #3b82f6;
--vgt-neon-cyan: #06b6d4;
--vgt-neon-gold: #f59e0b;
}

```

5.2 Матрица привязки stylesheets

```

assets/css/vis-dashboard.css
└─ Влияет на: весь GeDefense Dashboard (базовый layout, Topbar, Sidebar, Panel Grid)
└─ Подключается на: всех страницах 'admin.php?page=vgt-suite', 'vgt-throneguard', 'vgt-loginpager'

assets/css/vis-dashboard-modern.css
└─ Влияет на: Responsive Breakpoints (@media (max-width: 960px)), Accessibility (:focus-visible), prefers-reduced-motion
└─ Подключается на: всех Dashboard pages

assets/css/vis-scanner-modal.css
└─ Влияет на: modal прогресса сканирования и Code Inspector Explorer
└─ Подключается на: всех Dashboard pages (для глобального scanner client)

assets/css/vis-security-center.css
└─ Влияет на: Dashboard -> Security Center (Tabs: Assurance, System Status, Logs)
└─ Подключается на: Tab 'security_center'

assets/css/vis-titan.css
└─ Влияет на: Dashboard -> Titan Hardening
└─ Подключается на: Tab 'titan'

assets/css/vis-xdr.css
└─ Влияет на: Dashboard -> Trinity, Thread, XDR
└─ Подключается на: Tabs 'trinity', 'thread', 'xdr'

includes/dashboard/views/sidebar/style.css
└─ Влияет на: левую navigation bar, SVG icons, section dividers
└─ Подключается на: всех Dashboard pages

includes/dashboard/views/{tab}/style.css (27 module-specific stylesheets)
└─ aegis/style.css -> Tab 'aegis'
└─ airlock/style.css -> Tab 'airlock'
└─ cerberus/style.css -> Tab 'cerberus'
└─ chronos/style.css -> Tab 'chronos'
└─ downloads/style.css -> Tab 'downloads'
└─ filesystem/style.css -> Tab 'filesystem'
└─ ghost_trap/style.css -> Tab 'ghost_trap'
└─ gorgon/style.css -> Tab 'gorgon'
└─ hades/style.css -> Tab 'hades'
└─ integrity/style.css -> Tab 'integrity'
└─ kernel/style.css -> Tab 'kernel'
└─ loginpager/style.css -> Tab 'loginpager' & 'vgt-loginpager'
└─ logs/style.css -> Tab 'security_center' (section Logs)
└─ modules/style.css -> Tab 'modules'
└─ morpheus/style.css -> Tab 'morpheus'
└─ nemesis/style.css -> Tab 'nemesis'
└─ oracle/style.css -> Tab 'oracle'
└─ overview/style.css -> Tab 'overview'
└─ prometheus/style.css -> Tab 'prometheus'
└─ setup_wizard/style.css -> Tab 'setup_wizard'
└─ styx/style.css -> Tab 'styx'
└─ systatus/style.css -> Tab 'security_center' (section System)
└─ thread/style.css -> Tab 'thread'
└─ throneguard/style.css -> Tab 'throneguard' & 'vgt-throneguard'
└─ trinity/style.css -> Tab 'trinity'
└─ vault/style.css -> Tab 'vault'
└─ vlp/style.css -> Tab 'vlp'
└─ xdr/style.css -> Tab 'xdr'
└─ zeus/style.css -> Tab 'zeus'

```

6. API АРХИТЕКТУРА

6.1 AJAX endpoints (wp_ajax_*)

Action Name	Метод	Frontend Caller	Handler	Service / Class	Data Source / Action
vis_approve_changes	POST	integrity/script.js	handle_approve	VIS_Dashboard_Ajax	Обновляет Manifest Baseline
vis_save_zeus_config	POST	zeus/script.js	handle_zeus_config	Zeus_Config_Repository	Сохраняет WAF configuration
vis_zeus_run_benchmark	POST	zeus/script.js	handle_zeus_benchmark	Zeus_Benchmark	Выполняет 1000 итераций WAF regex
vis_zeus_run_self_test	POST	zeus/script.js	handle_zeus_self_test	Zeus_Contracts	Валидирует WAF signatures
vis_zeus_drain_blackbox	POST	zeus/script.js	handle_zeus_drain_blackbox	Zeus_Blackbox	Очищает WAF telemetry buffer
vis_zeus_restore_preset	POST	zeus/script.js	handle_zeus_restore_preset	Zeus_Config_Repository	Восстанавливает default WAF values
vis_zeus_save_contract	POST	zeus/script.js	handle_zeus_save_contract	Zeus_Contracts	Сохраняет WAF rule contract
vis_zeus_delete_contract	POST	zeus/script.js	handle_zeus_delete_contract	Zeus_Contracts	Удаляет WAF rule contract
vis_zeus_generate_admission_token	POST	zeus/script.js	handle_zeus_generate_token	Zeus_Admission	Создает подписанный WAF bypass token
vis_zeus_rollback_policy	POST	zeus/script.js	handle_zeus_rollback_policy	Zeus_Config_Repository	Выполняет rollback к предыдущей policy
vis_dashboard_unban_ip	POST	cerberus/script.js, vis-dashboard.js	handle_unban_ip	VIS_Cerberus	Удаляет IP из wp_vis_apex_bans
vis_run_scan	POST	vis-scanner-client.js	handle_scan_bridge	VIS_Scanner_Engine_Omega	Выполняет batch file scan
vgt_integrity_uplink	POST	vis-scanner-client.js	handle_scan_bridge	VIS_Scanner_Engine_Omega	Alias для scan batch
vis_inspect_file	POST	integrity/script.js	handle_inspect_file	VIS_Dashboard_Ajax	Читает source file внутри Jailed Path
vis_security_center_test	POST	vis-security-center.js	handle_security_center_test	VIS_Security_Center	Выполняет host & kernel diagnostics
vis_oracle_ping	POST	vis-oracle-diagnostics.js	handle_oracle_ping	VIS_Oracle	Ping test WAF & DB
vis_upload_addon	POST	modules/script.js	handle_upload_addon	VIS_Module_Registry	Проверяет и распаковывает ZIP Add-On
vis_uninstall_addon	POST	modules/script.js	handle_uninstall_addon	VIS_Module_Registry	Удаляет dynamic Add-On directory
vgt_gorgon_toggle	POST	gorgon/script.js	handle_toggle	Gorgon_Ajax	Включает/выключает Mesh Sync
vgt_gorgon_update_config	POST	gorgon/script.js	handle_update_config	Gorgon_Ajax	Сохраняет Nexus URL & API Key
vgt_gorgon_ping_nexus	POST	gorgon/script.js	handle_ping_nexus	Gorgon_Uplink	Отправляет ping на remote Nexus server
vgt_gorgon_sync	POST	gorgon/script.js	handle_sync	Gorgon_Sync_Engine	Выполняет ad-hoc Mesh synchronization
vgt_gorgon_add_node	POST	gorgon/script.js	handle_add_node	Gorgon_Config	Регистрирует новый peer node
vgt_gorgon_remove_node	POST	gorgon/script.js	handle_remove_node	Gorgon_Config	Удаляет peer node

Action Name	Метод	Frontend Caller	Handler	Service / Class	Data Source / Action
vgt_morpheus_trigger_ai	POST	morpheus/script.js	ajax_trigger_ai	Morpheus_AI	Запускает AI callstack analysis
vgt_morpheus_reject_ai	POST	morpheus/script.js	ajax_reject_ai	Morpheus_Dashboard	Отклоняет AI proposal
vgt_morpheus_approve_ai	POST	morpheus/script.js	ajax_approve_ai	Morpheus_Dashboard	Принимает AI rule
vgt_morpheus_delete_matrix	POST	morpheus/script.js	ajax_delete_matrix	Morpheus_Dashboard	Очищает Audit Matrix
vgt_morpheus_toggle_strict	POST	morpheus/script.js	ajax_toggle_strict	Morpheus_Dashboard	Переключает RASP Strict Mode
vis_throneguard_clear_logs	POST	throneguard/script.js	handle_clear_logs	VIS_Throne_Guard	Очищает ThroneGuard Audit Log
vg_seo_generate	POST	VisionGaiaSEO UI	handle_generate_seo	VG_Automation_Engine	Генерирует SEO metadata через AI
vg_seo_reset_stats	POST	VisionGaiaSEO UI	handle_reset_stats	VG_SEO_Ajax_Router	Сбрасывает SEO counters
vgt_add_redirect	POST	VisionGaiaSEO UI	handle_add_redirect	VGT_Redirect	Создает 301 redirect
vgt_delete_redirect	POST	VisionGaiaSEO UI	handle_delete_redirect	VGT_Redirect	Удаляет 301 redirect
vgt_render_preview	POST	builder/views/editor-ui.php	ajax_render_preview	VGT_Ajax	Рендерит Live Preview
vgt_copilot_generate	POST	builder/views/editor-ui.php	ajax_copilot_generate	VGT_Ajax	Генерирует UI component через AI
vgt_migrate_elementor	POST	builder/views/bulk-migration.php	ajax_migrate	VGT_Migration	Мигрирует Elementor post
vlp_download_asset_secure	POST	vlp/script.js	handle_ajax_download	VLP_Asset_Downloader	Загружает asset в local proxy

6.2 Admin-Post actions (admin_post_*)

Action Name	Метод	Caller	Handler	Class	Функция
vgt_export_sentinel_data	POST	view-security_center.php	handle	VIS_Sentinel_Export	Генерирует pseudonymized JSON export
vis_download_register	POST	view-downloads.php	registerAction	DownloadManager	Создает защищенную file сопу
vis_download_toggle	POST	view-downloads.php	toggleAction	DownloadManager	Приостанавливает/активирует download link
vis_download_delete	POST	view-downloads.php	deleteAction	DownloadManager	Удаляет защищенную file сопу
vis_throneguard_claim	POST	view-throneguard.php	handle_claim_master	VIS_Throne_Guard	Назначает Master Admin role
vis_throneguard_save	POST	view-throneguard.php	handle_save	VIS_Throne_Guard	Сохраняет role permissions
vis_throneguard_unlock	POST	view-throneguard.php	handle_unlock	VIS_Throne_Guard	Валидирует Superkey и unlock session
vis_titan_policy_action	POST	assets/js/vis-titan-command-center.js	handle_policy_action	VIS_Titan	Выполняет transition между stages (Test/Enforce/Rollback)

Action Name	Метод	Caller	Handler	Class	Функция
vis_titan_download_nginx	POST	view-titan.php	handleDownload	VIS_Titan_Server_Rules	Генерирует Nginx configuration
vis_titan_generate_gate_link	POST	assets/js/vis-titan-command-center.js	handleGenerateLink	VIS_Titan_Login_Gate	Создает one-time login link
vis_titan_preview_link	POST	assets/js/vis-titan-command-center.js	handlePreviewLink	VIS_Titan_Sandbox	Создает secure Sandbox Preview URL
vgt_sync	POST	view-vlp.php	process_live_sync	VLP_Admin_Dashboard	Выполняет VLP Asset Synchronization
vgt_export_csv	POST	view-vlp.php	stream_csv	VLP_Admin_Dashboard	Экспортирует Consent Logs в CSV
vgt_export_pdf	POST	view-vlp.php	render_print_view	VLP_Admin_Dashboard	Создает print view для compliance audits
vgt_vault_save	POST	VisionGaiaSE0 UI	handle_save	VG_Vault_Bridge	Безопасно сохраняет API keys в Vault
vgt_vault_delete	POST	VisionGaiaSE0 UI	handle_delete	VG_Vault_Bridge	Удаляет API keys из Vault

6.3 REST API endpoints

Route	Метод	Authentication	Handler	Class	Назначение
/wp-json/visiongaia/v1/titan/csp-report	POST	Her (<code>__return_true</code>)	collect	VIS_Titan_Violation_Collector	Принимает standardized CSP violation reports от browser

7. ПОТОКИ ДАННЫХ

7.1 Ingress Request Inspection & Attack Drop Pipeline

```

[ Входящий HTTP Request ]
↓
[ Phase 1: Pre-Flight Kernel ]
↓
1. Cerberus: 0(1) Memory Lookup (APCu)
   └─ IP заблокирован? -> HTTP 403 (завершение < 0.1 ms, 0% DB Load)
   └─ IP не заблокирован -> Далее
↓
2. Zeus: Pre-Boot 6G DFA Match
   └─ Вредоносный Query String / Bad Bot? -> HTTP 403 Drop & Telemetry Logging
   └─ Clean -> Далее
↓
3. Aegis: Deep Packet Inspection (DPI)
   └─ Нормализация (SQL comments, Unicode, Quote-Slashing)
   └─ Рекурсивная проверка tokens (GET, POST, Headers, JSON)
   └─ Exploit обнаружен? -> HTTP 403 + Event в EventBus + Ван через Cerberus
   └─ Clean -> Далее
↓
4. Prometheus: Behavioral Scoring
   └─ Накопление subnet/IP score и проверка Webshell heuristics
   └─ Threshold превышен? -> Ван через Cerberus
   └─ Clean -> Далее
↓
5. Nemesis: Deception Check
   └─ Request направлен на decoy route? -> Bounded Tarpit Delay + Fake Response
   └─ Нормальный traffic -> Далее
↓
6. Morpheus: RASP Hypervisor
   └─ Проверяет DML protections для wp_users и SSRF на 169.254.169.254
   └─ Нарушений нет -> Далее
↓
[ Phase 2: WordPress Normal Boot (plugins_loaded / init) ]
↓
Titan: Surface Policy Injection (CSP, HSTS, Permissions-Policy)

```

↓
Отправка response клиенту

7.2 TRINITY XDR Incident Correlation & Response Flow

```

Security Event на sensor (например Aegis WAF SQLi Detection)
↓
VIS_Event_Bus::log() & EventFabric::ingest()
↓
Создание XdrEvent с Actor Hash & Correlation ID
↓
EventRepository::persist() -> сохранение в wp_vis_xdr_events
↓
IncidentEngine::correlate()
↓
Есть активный incident для того же actor/hash?
├─ ДА -> связь в wp_vis_xdr_incident_events, обновление Incident
└─ НЕТ -> создание нового incident в wp_vis_xdr_incidents + Attack Story
↓
PolicyEngine::evaluate() -> проверка детерминированных response rules
↓
ResponseEngine::execute()
↓
Countermeasure (например Cerberus IP Ban или Session Kill)
↓
Сохранение в wp_vis_xdr_responses (включая Rollback JSON)
↓
EvidenceStore::append() -> Merkle Digest chain в wp_vis_xdr_evidence

```

7.3 Integrity Scan & Baseline Commit Flow

```

Пользователь нажимает "Запустить сканирование" или срабатывает Chronos Cron
↓
AJAX 'vis_run_scan' -> VIS_Dashboard_Ajax::handle_scan_bridge()
↓
VIS_Scanner_Engine_Omega::run_batch()
↓
Итерация путей в Webroot с учетом vis-vault-omega/
↓
Для каждого файла:
├─ вычислить SHA-256 hash
├─ VIS_Malware_Engine::inspect() (AST-Lexical, Polyglot, SVG)
└─ сравнить с локальной Baseline (NDJSON)
↓
Обнаружено изменение или malware?
├─ ДА -> создать Finding, при необходимости изолировать в VIS_Quarantine_Store
└─ НЕТ -> файл без изменений
↓
Обновить State JSON в vis-vault-omega/
↓
Вернуть Batch Result как JSON в vis-scanner-client.js
↓
[Опционально] пользователь нажимает "Подтвердить изменения"
↓
AJAX 'vis_approve_changes'
↓
Пересчет Merkle Root & Commit в integrity_matrix.json

```

8. SHARED / CORE ФАЙЛЫ

Следующие файлы образуют **обязательный фундамент** Suite. Они классифицированы как CORE / SHARED DEPENDENCY:

1. class-vis-bootstrapper.php

- **Class:** VIS_Bootstrapper
- **Role:** Master Kernel Bootstrapper.
- **Используется в:** gedefense-wp.php.
- **Зависимые модули:** все 20 modules и subsystems инициализируются Bootstrapper.

2. class-vis-schema.php

- **Class:** VIS_Schema
- **Role:** оркестрация Database & Vault directory.
- **Используется в:** gedefense-wp.php (Activation Hook и admin_init Upgrade Check).
- **Зависимые модули:** Cerberus, XDR, Secure Downloads, Oracle, Rate Limiting.

3. class-vis-vault.php

- **Class:** VIS_Vault

- **Role:** low-level cryptography engine (Libsodium / OpenSSL AES-256-GCM).
- **Используется в:** VIS_Key_Vault, VIS_Throne_Guard, VIS_Nemesis, VIS_Titan, VG_Vault_Bridge.

4. includes/core/class-vis-security.php

- **Class:** VIS_Security
- **Role:** центральная Zero-Trust utility library (client_ip, jailed_path, pinned_https_get, timing_safe_equals).
- **Используется в:** Cerberus, Aegis, Zeus, Prometheus, Styx, Hades, Titan, Airlock, Dashboard Core.

5. includes/core/class-namespace-compatibility.php

- **Class:** VisionGaia\GeDefense\Core\NamespaceCompatibility
- **Role:** Autoloader & interface bridge между canonical namespace и legacy class names.
- **Используется в:** VIS_Bootstrapper::register_autoloader().

6. includes/core/class-vis-event-bus.php

- **Class:** VIS_Event_Bus
- **Role:** стандартизированный Security Event Emitter.
- **Используется в:** Aegis, Cerberus, Prometheus, Nemesis, Ghost Trap, Airlock, Styx.

7. includes/core/class-vis-security-health.php

- **Class:** VIS_Security_Health
- **Role:** проверка environment и PHP stability.
- **Используется в:** Security Center, Overview Cockpit, Oracle, Kernel Sentinel.

8. includes/dashboard/class-vis-dashboard-assets.php

- **Class:** VIS_Dashboard_Assets
- **Role:** детерминированный Enqueue Manager для styles и scripts.
- **Используется в:** VIS_Dashboard_Core::inject_assets().

9. includes/xdr/class-xdr-event-fabric.php

- **Class:** VisionGaia\GeDefense\Xdr\EventFabric
- **Role:** центральный backbone для Incident Correlation и Response Triggers.
- **Используется в:** EventBus и всех sensor modules.

9. АРХИТЕКТУРНЫЕ СВЯЗИ (MERMAID DIAGRAMS)

9.1 Архитектура всей системы

```
graph TD
    Client[HTTP Request] --> PluginEntry[gedefense-wp.php]
    PluginEntry --> Bootstrapper[VIS_Bootstrapper]

    subgraph "Phase 1: Pre-Flight Kernel (Zero-Overhead)"
        Bootstrapper --> Cerberus[Cerberus L0 Drop]
        Cerberus --> Zeus[Zeus 6G WAF]
        Zeus --> Aegis[Aegis DPI WAF]
        Aegis --> Prometheus[Prometheus Profiler]
        Prometheus --> Nemesis[Nemesis Deception]
        Nemesis --> Morpheus[Morpheus RASP]
        Morpheus --> Gorgon[Gorgon Telemetry]
    end

    subgraph "Phase 2: Invariant Subsystems (plugins_loaded)"
        Titan[Titan Surface Policy]
        Hades[Hades Admin Cloaking]
        Airlock[Airlock Upload Guard]
        Styx[Styx Outbound Shield]
        ThroneGuard[ThroneGuard Superkey Gate]
        Chronos[Chronos Autopilot]
        Vault[Key Vault KMS]
    end

    Morpheus --> WP[WordPress Application Runtime]
    WP --> Phase2[Phase 2 Invariants]
    Phase2 --> Titan
    Phase2 --> Hades
    Phase2 --> Airlock
    Phase2 --> Styx
    Phase2 --> ThroneGuard
    Phase2 --> Chronos
    Phase2 --> Vault
```

```

subgraph "Data & Security Backbone"
    XDR[TRINITY XDR Engine]
    EventBus[VIS_Event_Bus]
    Schema[(MySQL DB: 10 Tables)]
    VaultDir[(Sovereign Vault Storage)]
end

Aegis --> EventBus
Cerberus --> EventBus
Prometheus --> EventBus
EventBus --> XDR
XDR --> Schema
Chronos --> VaultDir
Vault --> VaultDir

```

9.2 Dashboard Architecture

```

graph TD
    User[Administrator] --> Core[VIS_Dashboard_Core]
    Core --> MenuMatrix[register_menu_matrix]
    MenuMatrix --> Suite[admin.php?page=vgt-suite]
    MenuMatrix --> ThronePage[admin.php?page=vgt-throneguard]
    MenuMatrix --> LoginPage[admin.php?page=vgt-loginpager]

    Suite --> ViewRouter[VIS_Dashboard_View::render]
    ViewRouter --> Sidebar[view-sidebar.php]
    ViewRouter --> ActiveTab{Tab Switcher}

    ActiveTab --> TabOverview[view-overview.php]
    ActiveTab --> TabZeus[view-zeus.php]
    ActiveTab --> TabAegis[view-aegis.php]
    ActiveTab --> TabTitan[view-titan.php]
    ActiveTab --> TabTrinity[view-trinity.php]
    ActiveTab --> TabSecurity[view-security_center.php]
    ActiveTab --> TabOthers[21 других Views...]

    Core --> AssetManager[VIS_Dashboard_Assets::enqueue]
    AssetManager --> GlobalCSS[vis-dashboard.css / modern.css]
    AssetManager --> TabCSS[Tab-specific style.css]
    AssetManager --> TabJS[Tab-specific script.js]

    TabJS --> AjaxHub[VIS_Dashboard_Ajax]
    AjaxHub --> BackendHandlers[Backend Services & Scanners]

```

9.3 Module Dependencies

```

graph LR
    subgraph "Core Utilities"
        Security[VIS_Security]
        VaultCore[VIS_Vault]
        Health[VIS_Security_Health]
        EventBus[VIS_Event_Bus]
    end

    subgraph "Enforcement & Defense"
        Cerberus[Cerberus]
        Aegis[Aegis]
        Zeus[Zeus]
        Prometheus[Prometheus]
        Nemesis[Nemesis]
        GhostTrap[Ghost Trap]
        Airlock[Airlock]
        Styx[Styx]
        Titan[Titan]
        ThroneGuard[ThroneGuard]
    end

    Security --> Cerberus
    Security --> Aegis
    Security --> Zeus
    Security --> Styx
    Security --> Titan
    VaultCore --> ThroneGuard
    VaultCore --> Nemesis
    VaultCore --> Titan

    Aegis --> Cerberus
    Prometheus --> Cerberus
    GhostTrap --> Cerberus

    Aegis --> EventBus
    Zeus --> EventBus
    Prometheus --> EventBus
    EventBus --> XDR[TRINITY XDR]
    XDR --> Cerberus

```

9.4 Backend/API/Database Data Flow

```

sequenceDiagram
    autonumber
    participant Browser as Client / Attacker

```

```

participant WAF as Aegis / Zeus
participant EventFabric as TRINITY XDR EventFabric
participant Repo as EventRepository
participant Engine as IncidentEngine
participant DB as MariaDB / MySQL
participant Cerberus as Cerberus L0

Browser->>WAF: Malicious Request (SQLi / XSS)
WAF->>WAF: Recursive Normalization & DFA Match
WAF->>EventFabric: ingest(XdrEvent)
EventFabric->>Repo: persist()
Repo->>DB: INSERT INTO wp_vis_xdr_events
EventFabric->>Engine: correlate(event)
Engine->>DB: SELECT/UPDATE wp_vis_xdr_incidents
Engine->>Cerberus: trigger_ban(ip, reason)
Cerberus->>DB: INSERT INTO wp_vis_apex_bans
Cerberus->>Cerberus: Cache in APCu / RAM
WAF->>Browser: HTTP 403 Forbidden (Exploit Terminated)

```

10. ССЫЛКИ НА ФАЙЛЫ

Для однозначной технической прослеживаемости ключевые class definitions сопоставлены с их source paths:

Class / Interface	File	Relevant Methods
VIS_Bootstrapper	class-vis-bootstrapper.php	register_autoloader(), engage_phase_1(), engage_phase_2()
VIS_Schema	class-vis-schema.php	enforce(), XDR_SCHEMA_VERSION, DOWNLOAD_SCHEMA_VERSION
VIS_Vault	class-vis-vault.php	encrypt(), decrypt(), get_primary_key()
VIS_Security	includes/core/class-vis-security.php	client_ip(), pinned_https_get(), jailed_path()
NamespaceCompatibility	includes/core/class-namespace-compatibility.php	register(), autoload(), canonical()
VIS_Module_Integrity	includes/core/class-vis-module-integrity.php	verify_all(), calculate_merkle_root()
VIS_Module_Registry	includes/core/class-vis-module-registry.php	all(), enabled(), path()
VIS_Event_Bus	includes/core/class-vis-event-bus.php	init(), log(), emit()
VIS_Security_Health	includes/core/class-vis-security-health.php	evaluate(), score()
VIS_Security_Center	includes/core/class-vis-security-center.php	run_diagnostics(), get_snapshot()
VIS_Cerberus	includes/modules/cerberus/class-vis-cerberus.php	instance(), ban_ip(), unban_ip(), check_request()
VIS_Aegis	includes/modules/aegis/class-vis-aegis.php	__construct(), inspect(), normalize()
VIS_Zeus	includes/modules/zeus/class-vis-zeus.php	get_instance(), filter_request()
VIS_Prometheus	includes/modules/prometheus/class-vis-prometheus.php	evaluate_request(), record_threat()
VIS_Nemesis	includes/modules/nemesis/class-vis-nemesis.php	deploy_deception(), trap()
Vis_Morpheus	includes/modules/morpheus/class-vis-morpheus.php	engage(), get_hypervisor()
VIS_Titan	includes/modules/titan/class-vis-titan.php	__construct(), enforce_protocols(), handle_policy_action()
VIS_Throne_Guard	includes/modules/throneguard/class-vis-throne-guard.php	get_instance(), handle_claim_master(), handle_unlock()
DownloadManager	includes/modules/downloads/class-secure-download-manager.php	boot(), serve(), registerAction()

Class / Interface	File	Relevant Methods
VIS_Dashboard_Core	includes/dashboard/class-vis-dashboard-core.php	register_menu_matrix(), inject_assets()
VIS_Dashboard_View	includes/dashboard/class-vis-dashboard-view.php	get_tabs(), render()
VIS_Dashboard_Settings	includes/dashboard/class-vis-dashboard-settings.php	process_mutations(), handle_standard_config()
VIS_Dashboard_Assets	includes/dashboard/class-vis-dashboard-assets.php	enqueue()
VIS_Dashboard_Ajax	includes/dashboard/class-vis-dashboard-ajax.php	mount_endpoints(), handle_scan_bridge()
EventFabric	includes/xdr/class-xdr-event-fabric.php	boot(), ingest(), record_event()
VIS_Scanner_Engine_Omega	includes/scanner/class-vis-scanner-engine.php	run_batch(), commit_baseline()

11. АРХИТЕКТУРНЫЕ ЗАМЕЧАНИЯ

В ходе анализа кода были выявлены следующие архитектурные особенности:

1. Неактивные / исторические Views в Dashboard directory:

- `includes/dashboard/views/view-aos.php`: Bridge View для All-In-One Security (AIOS), не зарегистрированный в активном Tab Router `$tabs`. Все еще содержит inline `<style>` tag.
- `includes/dashboard/views/view-myrmidon.php`: неиспользуемый Device Trust View с inline event handlers (`onclick`, `onsubmit`), который не подключен к menu routing.
- **Рекомендация**: в среднесрочной перспективе перенести эти файлы в архив `legacy/` либо полностью удалить, чтобы уменьшить code ballast.

2. Binary archive artifacts внутри source tree:

- `includes/modules/gorgon/gorgon.tar.gz` (7,251 bytes)
- `includes/VisionGaiaSE0/VisionGaiaSE0.zip` (43,980 bytes)
- **Рекомендация**: хранение ZIP/TAR archives в PHP source tree следует заменить внешним release/package manager.

3. Несоответствие структуры tests:

- Directory `tests/Unit/` пуст.
- Вся test coverage проекта (28 специализированных regression tests для WAF, XDR, Titan, Scanner, Morpheus) находится в directory `scripts/`.
- **Рекомендация**: test suite в `scripts/` полностью работоспособна и не зависит от PHPUnit; `tests/` можно очистить либо формально заполнить wrapper classes.

4. Сильные Cross-Module Dependencies:

- `VIS_Aegis` и `VIS_Prometheus` напрямую связаны с `VIS_Cerberus::ban_ip()`.
- `VIS_Zeus` имеет прямой bridge к XDR (`Zeus_Xdr_Bridge`).
- `VIS_Titan` читает quarantine certificates из `Airlock`.
- **Обоснование безопасности**: эти связи являются намеренными, поскольку на L0-L1 требуется прямая атомарная передача WAF decisions в blocking filter, чтобы минимизировать latency.

Документ сгенерирован и проверен для GeDefense WP - Open Core (Standard WordPress Plugin v8.1.0).

Часть V - Источники и версия

Это руководство создано на основе предоставленного исходного кода **GeDefense WP - Open Core 8.1.0** и его `ARCHITECTURE.md`. Описания настроек и функций относятся именно к этой версии. После последующих изменений кода необходимо проверять и обновлять как архитектурную карту, так и руководство.